Department of Computer Science

BSc (Hons) Computer Science (with Artificial Intelligence)

Academic Year 2025 - 2026

Comparative Evaluation of CNN Architectures for MSK Abnormality Detection: A Controlled Analysis of ResNet50 and EfficientNet-B0

Nathan Singh

A report submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science

Brunel University London
Department of Computer Science
Uxbridge
Middlesex
UB8 3PH
United Kingdom
F: +44 (0) 1895 251686

Abstract

This dissertation investigates the use of deep learning to detect musculoskeletal (MSK) abnormalities in radiographic images. Using this clinically relevant context as a practical setting for evaluating convolutional neural networks(CNN) architectures. MSK injuries represent a portion of hospital imaging workloads. Interpretation of radiographs is traditionally performed by trained doctors, this manual review of radiographs may involve large workloads, time pressures and variance in the availability of the specialists needed to carry out this task.

These factors make automated analysis of radiographs an important research area and provide an important context in which the behaviour of deep learning models like CNNs can be evaluated.

This dissertation focuses on a comparative evaluation of 2 models belonging to families of widely used CNN architectures, ResNet50 and EfficientNet-B0. The comparison will be carried out on the Musculoskeletal Radiographs (MURA) dataset by Stanford University. MURA is one of the largest publicly available datasets of MSK images. The dataset is widely used as a benchmark for abnormality detection and was originally designed used in a competition hosted by Stanford to compare deep learning models against six board certified radiologists. This highlights the difficulty and real world relevance of this task.

A controlled experimental framework was developed using PyTorch to create a fair architectural comparison. Both models were initialised using ImageNet pretrained weights and trained using identical pipelines and evaluation processes. Dataset partitioning was performed at the patient level to prevent data leakage, experiments were conducted across a couple of dataset splits and seeds to analyse the models. Model performance was evaluated using a range of metrics at image and study level with an emphasis on abnormal cases. Grad-CAM was also implemented to generate visualisations of how a model comes to its conclusions.

In summary, this analysis will provide a structured comparison between a more traditional residual network architecture and a more modern parameter efficient architecture to demonstrate how architecture designs influences performance, stability and interpretation when applied to MSK images.

Results show consistent differences in classification performance and varied training behaviour between the 2 models, modern efficiency focused CNNs like EfficientNet-B0 demonstrate a stronger ability to recognise abnormal cases.

Acknowledgements

Above all I thank God for His strength. To my parents, thank you for your unwavering support and ensuring my focus was only my education; I owe this entire chapter of my educational life to your sacrifices. I am more than grateful for A. You were the silent engine behind this work, by my side every single day and refusing to let me settle for anything other than my best.

I certify that the work presented in the dissertation is my own unless referenced.

Signature: Nathan Singh

Date: 26/03/2026

Total Words: 20480

Table of Contents

List of Tables

List of Figures

1 Introduction

1.1 Introduction

Musculoskeletal (MSK) disorders represent one of the most significant contributors to global disability burden, it affects around 1.7 billion people worldwide and accounts for the largest share of years lived with a disability (GBD 2021 Other Musculoskeletal Disorders Collaborators, 2023). These conditions are projected to increase over coming decades, mostly due to ageing populations and rising rates of comorbid conditions. This places a pressure on radiological services that already operate consistent load. Radiographs remain the primary imaging mode for initial assessment of MSK abnormalities and their interpretation requires specific medical expertise that is not available across all healthcare settings (The Royal College of Radiologists , 2023). This combination of scale, complexity and limited resources makes automated support for radiograph analysis a meaningful research area.

Medical imaging plays an important role in the medical world, especially in the assessment of MSK injuries. Radiographs are widely used detect fractures, joint abnormalities and other skeletal issues. However, interpreting radiographs is traditionally performed manually by trained clinicians. This motivates research and a need for computational approaches capable of assisting radiograph analysis.

Recently, deep learning methods have achieved success in image recognition task, mainly through the use of CNNs. These models are capable of learning visual features directly from an image, therefore becoming a dominant approach in many applications, including in the medical realm (Litjens, et al., 2017). Due to this, there is an expanding interest in how different CNN architectures perform when applied to radiographic imaging tasks.

One of the most widely used benchmarks to address this problem is the MURA dataset which contains over 40,000 images (Rajpurkar, et al., 2018) . The original purpose of MURA was a competition where contestants aimed to build the best model to detect abnormalities at the study level. Their scores were compared against six board certified radiologists tasked with the same. Although the best model demonstrated competitive scores on certain studies, the radiologists still achieved higher scores, this highlights the difficulty of the task and further supports MURA as a challenging benchmark for evaluating architectural differences.

This dissertation uses MURA as a controlled environment for evaluating 2 models, ResNet50 and EfficientNet-B0 which represent different philosophies within deep learning; depth driven

residual learning against compound scaling parameter efficiency. By training and evaluating both models under the same conditions, the project will investigate how architecture informs performance.

1.2 Aims and Objectives

Aim:

The aim of this dissertation is to carry out a controlled comparative analysis of 2 CNN architectures: ResNet50 and EfficientNet-B0, for the detection of MSK abnormalities using the MURA dataset. This is done to investigate how differences in CNN architectures influence abnormality detection performance, model stability and interpretability in radiographs.

Objectives:

1) Conduct a background research on deep learning methods used in medical image analysis, with particular focus on CNNs, radiography classification and assess existing research in and around the MURA dataset.
2) Design a controlled experimental framework that allows for fair comparison between CNN architectures by making sure of similarity between preprocessing pipelines, augmentations, training procedures and evaluation circumstances.
3) Implement and train ResNet50 and EfficientNet-B0 using transfer learning with ImageNet pretrained weights.
4) Assess how dataset partitioning strategy influences architectural comparisons.
5) Evaluate performance using appropriate metrics at both image and study levels.
6) Investigate the interpretability of each architecture's decision making process to investigate whether predictions align with relevant features.

1.3 Project Approach

This dissertation follows a structured approach to enable a controlled comparison between 2 CNNs applied to musculoskeletal radiograph classification. This dissertation is organised into chapters that progressively develop the research problem, implement the experimental design and finally evaluate the 2 models used in this study.

The background chapter provides theoretical and technical context needed for the study. It looks into existing research in deep learning for a medical context. The chapter also introduces the MURA dataset and discusses previous work using this dataset, including the original study which compared best model designs and their scores against board certified radiologists. The background section establishes motivation for comparing architectures in this context.

Following the background chapter, the methodology describes the experimental design used in this project. It explains the reasons behind the comparative evaluation of the models in this study and outlines the training framework used to ensure fairness in comparisons. Important topics discussed include dataset preparation, patient level dataset splitting to avoid leakage, use of transfer learning via ImageNet weights and the implementation of preprocessing and data augmentations.

The design and implementation chapter goes into further detail about the content mentioned in the methodology section. It delves deeper into software architecture, individual pipelines, model implementation, training procedures and the mechanisms used for reproducible results through seeds and dataset splits.

The results chapter presents the outcomes of the experimental evaluation carried out on both models. Model performance is analysed using multiple metrics at both image and study levels. In addition, Grad-CAM visualisations are interpreted in this section to understand how a model assesses an image and comes to its predictions.

The conclusion chapter summarises the main findings of this dissertation and reflects on the extent to which the aims and objectives were achieved, it also discusses the limitations of current work and outlines potential future directions this project can branch into.
The structured progression of background research and theory to evaluation ensures that final conclusions can be interpreted within a defined framework.

1.4 Dissertation Outline

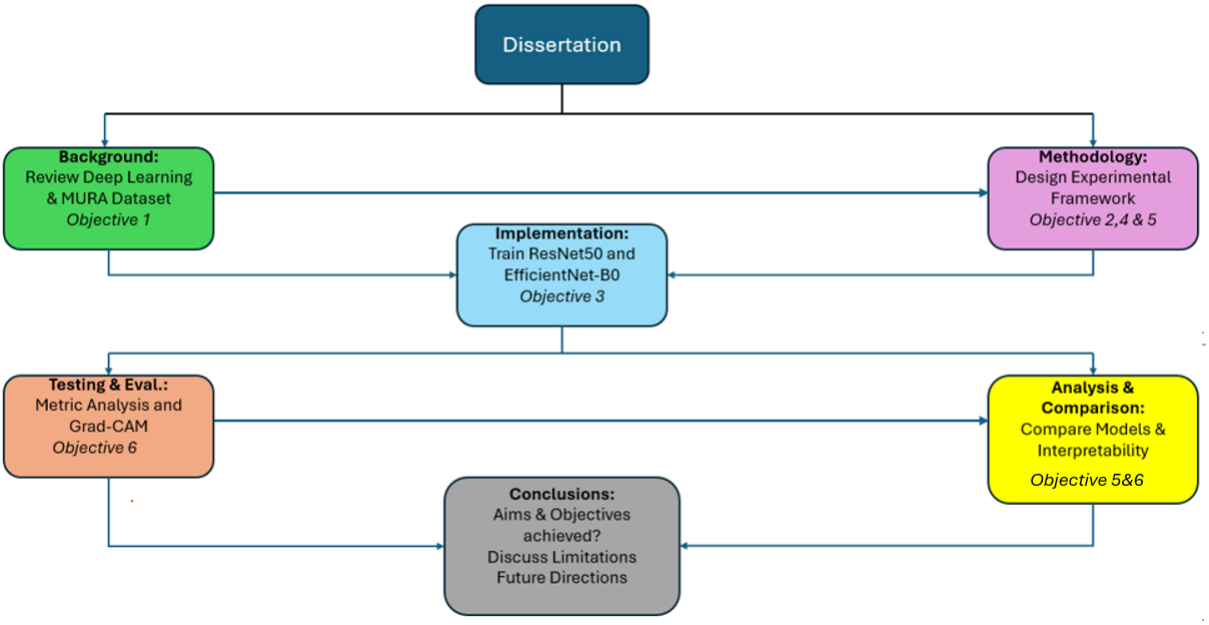


Figure 1.1: High Level Image Description of Dissertation and Workflow

The entire structure of this project is shown in Figure 1.1. The diagram highlights the flow of the dissertation from background research through to evaluations and final conclusions. It demonstrates how each stage of the project contributes to development, analysis and comparison of the 2 models and how outputs of each stage play into subsequent stages. The Figure above provides a clear overview of how this study is organised and how different individual chapters relate to the stated aims and objectives.

2 Background

2.1 Problem Context: Automated Musculoskeletal Radiograph Analysis

This project investigates how 2 different Convolutional Neural Network(CNN) architectures behave when applied to musculoskeletal(MSK) radiographs, using the MURA dataset as its basis. Although MSK imaging originates from a medical background, the focus of this project is a computer science problem, the project illustrates how architectural design choices influence model performance, generalisation and evaluation results. These tests will be carried out under an identical training pipeline to ensure that each model's behaviour can be attributed to its architecture as opposed to nuances in training.

Traditionally, MSK image interpretation is performed manually by medical experts, which is reliable but not an easily scalable task. Between a large number of images, varying opinions and subtle abnormalities; which if misdiagnosed can have catastrophic consequences (Burt, et al., 2017), this creates an environment where automated support systems could provide value. From a computational perspective, radiographs create challenging grounds for testing. Images often contain high resolution structures, subtle patterns and unstructured noise, this requires models to derive meaningful features from variations in intensity alone without the colour information that can be derived from natural images. This creates for a rich testbed to evaluate the strengths and weaknesses of different CNN architectures.

These factors motivate a comparative study rather than a single model improvement strategy. ResNet50 and EfficientNet-B0 were selected not simply because one is newer than the other, but because they represent 2 contrasting design principles; deep residual learning versus compound scaled parameter efficiency. This makes both models suitable for investigating how architectural choices play a role in behaviour under controlled conditions.

2.2 Dataset Selection and Characteristics

Although this project is not a medical study, the choice of dataset plays an important role in determining whether architectural differences can be meaningfully compared. The MURA dataset (Rajpurkar, et al., 2018) was released by Stanford University and is one of the largest publicly available collections of MSK radiographs and has become a benchmark for evaluation deep learning models on real world imaging tasks. It contains over 40,000 images grouped into studies, each labelled as normal or abnormal. The dataset structure introduces a level of complexity not present in simpler image classification datasets. MURA is valuable because it exposes models to challenges that test architectural strengths and weaknesses such as minor

abnormalities, high res grayscale images, varying positioning, etc. Datasets like CIFAR and ImageNet were avoided because they lack the grayscale complexity, class factor and image irregularities present in real medical data. These factors are important for comparing CNN architectures in a meaningful way.

A notable characteristic of this dataset is that the labels are assigned at the study level, unlike most datasets where labels are assigned at the image level. This introduces an extra layer of complexity not present in standard datasets. It requires assessment of how model architectures behave when predictions must be combined. It also makes a more realistic evaluation setting given its clinical relevance; doctors pass judgement on a patients wellbeing based of their whole scenario(study) rather than a singular image.

2.3 Convolutional Neural Networks in Medical Imaging

CNNs have a commonly used approach for image based tasks, this is because of their ability to automatically learn hierarchical visual features (Lin, et al., 2024). Earlier layers usually capture low level patterns such as edges, dots, lines, textures. Layers further down in the model collect more complex and meaningful abstract features. This behaviour is widely documented in deep learning literature (Lin, et al., 2024). This representation makes CNNs a good fit for radiography images.

In this context of MSK radiographs where features range from basic intensities to complex patterns. CNNs offer an efficient manner to analyse a large number of high resolution images consistently. Medical imaging reviews highlight that CNN systems have become important to computer aided diagnosis because of their strong performance and ability to generalise across different imaging formats (Mienye, et al., 2025). The absence of colour information in radiographs means that minor abnormalities need models to derive meaningful features from grayscale intensity variation. This places a greater importance on analysing architectural design, depth, and parameter efficiency. All of which dictate how effectively a model can learn from such data. These design features still differ between models, this can lead to distinct learning behaviours even when training processes are identical. Understanding these differences is imperative to this dissertation, commonly used CNNs in this context are VGG, DenseNet, Inception, MobileNet, ResNet, EfficientNet, etc. Each of these differ in depth, parameter efficiency and feature extraction strategies (Mienye, et al., 2025). ResNet and EfficientNet were selected for this study because they represent influential and contrasting designs in the form of deep residual learning and compound scaling efficiency respectively, making them ideal for a controlled comparison (Kailani, 2025)

2.4 ResNet Architectures

ResNet models represent a big change in deep learning by addressing the difficulty of training very deep neural networks (He, et al., 2015). Traditionally, CNNs suffer from the vanishing gradient problem and the degradation problem as depth increases (Bengio, et al., 1994). ResNet solves these problems via residual connections or skip connections which let layers learn modifications via identity mapping rather than a full transformation. In essence, traditional CNNs try to reinvent inputs on each layer, ResNet assumes input 'x' from the previous layer is strong, so instead of reinventing x, the layer only learns a variation F(x) to that input. These skip connections(the pathway that allows data to bypass) enable information and gradients to flow through the network more easily. Easily making it possible for models with even up to hundreds of layers. This innovation was first introduced in 2015 and rapidly became crucial in deep learning after achieving record breaking results (He, et al., 2015) on the ImageNet benchmark.

The ResNet family includes many models of varying depths such as ResNet18, ResNet34, ResNet101, ResNetRS and ResNet50, which is the model used in this study. It represents an established balance between depth and computational efficiency, deep enough to capture complex radiographic features while remaining feasible to train across multiple runs. It is also one of the most widely benchmarked models in medical imaging literature (He, et al., 2015).
It consists of 50 layers built from bottleneck residual blocks. Each of those 50 blocks uses a 1*1, 3*3, 1*1 convolution structure. This reduces computational cost while using fewer resources than older, shallower and less complicated architectures like VGG-16. ResNet50 is suitable for this project because it represents a 'classic' , well understood and widely benchmarked deep architecture. Its residual connections influence how it learns the subtleties of an image, how quickly it converges and how robust it is to image noise or variations in distribution, all of which are well documented in literature (He, et al., 2015). By comparing it to a more parameter efficient architecture such as EfficientNet-B0, it becomes possible to ascertain how differently designed CNN architectures perform on a dataset.

2.5 EfficientNet Architectures

Instead of increasing depth or width independently, EfficientNet uses compound scaling (Tan & Le, 2019). A method by which network depth, width and input resolution are scaled uniformly only using a single coefficient. It was demonstrated that balancing these 3 factors led to much better accuracy to efficiency trade-offs (Tan & Le, 2019) compared to altering any one of the three factors by itself. This approach allows EfficientNet models to achieve strong performance while remaining computationally lightweight.

The EfficientNet models range from EfficientNet-B0 to EfficientNet-B7, each variant of the model represents a larger and more computationally expensive model. Variant B0 was selected for this study because it is the baseline model of the family, it is the smallest and most parameter efficient, making it the most appropriate to compare against ResNet50, using larger scale models like variant B3 or B7 would have increased the time and resources needed to train on, the comparison would also be extremely biased, which would have reduced the feasibility of the design.

EfficientNet-B0 is built around MBConv; inverted residual blocks and Squeeze and Excitation attention mechanisms(SE). MBConv uses depth wise and separable convolutions to reduce computational cost. The SE layers adapt according to channel wise feature responses, improving representational quality without adding substantial computational costs. These architectural components allow EfficientNet-B0 to extract meaningful components using much fewer parameters (millions fewer) than deep architectures like the ResNet line of models.

EfficientNet is relevant for this project because it encapsulates a modern, more efficiency centred approach to CNN design. Their differences are larger than parameter count, they are unique in the method by which they diverge, extract features, how gradients propagate during training and how each model addresses the challenges of grayscale medical imaging.

2.6 Comparative Studies of CNN Architectures

Comparative studies of CNN architectures are well established in medical imaging (Mienye, et al., 2025). These studies reflect the broader challenge of selecting models that balance many factors including accuracy, robustness, efficiency. Many pieces of work have compared ResNet models to EfficientNet models across a variety of image categories. In short, this illustrates that architectural differences can lead to meaningful differences in performance.

For example, (Gulbahor, 2025) compared ResNet and EfficientNet across X-ray, CT scans and MRI datasets reporting that EfficientNet consistently achieved higher accuracy overall. ResNet's scores remained competitive, however the difference in the number of parameters was quite substantial. The paper referenced above also stated that 'EfficientNet kept computational costs lower than ResNet, making it ideal for hospitals or clinics with limited computing resources'. Similar findings appear in (Kumar, et al., 2024) lung cancer classification research where hybrid models were created by combining elements of ResNet50-101 and EfficientNet-B3. The hybrid model was shown to improve predictive performance and also highlighted complementary strengths between residual connections(ResNet architecture) and compound

scaling(EfficientNet architecture). Another study on CT scanned lung images (Sandag & Kabo, 2024) showed that both models performed particularly well under transfer learning, however once again EfficientNet provided better efficiency. The report also showed that ResNet offered stable convergence and strong baseline performance.

Outside of the medical world, studies of image classification architectures reinforce the aforementioned findings. A study (Kailani, 2025) reports that ResNet and EfficientNet represent two of the most influential CNN families. The study also reported that EfficientNet reported better computational efficiency while ResNet provided more representational power due to its depth.

However, these findings carry important limitations that motivate this dissertation. Both (Gulbahor, 2025) and (Sandag & Kabo, 2024) highlight EfficientNet's advantage through overall accuracy without looking further into training stability, study level aggregation or performance on smaller classes. (Kumar, et al., 2024) demonstrated a hybrid architecture that combines elements of both models, however this contaminates each models individual contribution to the task and makes architectural comparison difficult. None of these studies use the MURA dataset, meaning that findings are drawn from imaging context with fundamentally different characteristics such as colour images, larger datasets and different classification criteria that do not apply to radiograph analysis. This limits the extent to which their conclusions can be applied to this dissertation.

All of these findings indicate that EfficientNet frequently achieves stronger accuracy scores. But comparative studies illustrate that architectural performance varies across datasets, tasks, goals and evaluation criteria. Therefore, no single architecture is the 'best' in all contexts and the conditions under which each architecture performs well under remains an open question, especially in MSK analysis where dataset structure, class imbalance and evaluation priorities differ from standard benchmarks.

2.7 Existing Work on MURA

A small but relevant number of studies exist that apply CNNs to MURA, the original paper by (Rajpurkar, et al., 2018) established a DenseNet-169 baseline achieving a AUROC score of 0.929 with a sensitivity of 0.815 and specificity of 0.887 on the official, unreleased test set. This still remains as the most widely cited benchmark for MURA and is the standard against which models are compared against.

The (Teeyapan, 2021) paper investigated EfficientNet architectures on MURA and found that EfficientNet-B3 scored an overall Cohen's Kappa 0.717, outperforming MobileNetV2, DenseNet-169 and Xception as a baseline comparison. It is important to note that this study demonstrated that ImageNet pretrained weights require retraining on the full MURA training subset before fine tuning on individual study types, this finding informs the transfer learning strategy used in this dissertation. Another piece of research (Ananda, et al., 2021) compared 11 CNNs including ResNet50 specifically on the wrist radiographs from MURA. ResNet50 with augmentations achieved a mean accuracy of 0.857 and a kappa score of 0.703, Class Activation Mapping was applied to interpret model attention, it is similar to Grad-CAM analysis applied in this study. Another study used ResNet as 1 of 3 baseline architecture to illustrate competitive performance on the entire MURA dataset, although with significant variations across different body parts (He, et al., 2021).

Currently, there is no published study that directly compares ResNet50 and EfficientNet-B0 as individual architectures on the entire MURA dataset under controlled conditions with patient level splitting, multiple seeds and study level aggregation. The direct controlled comparison of these 2 models represents the primary gap this dissertation addresses.

2.8 Research Gap

Despite the volume of comparative work looking at these models, a number of limitations still exist. Many studies focus on overall accuracy only, without examining training processes, study level aggregations or interpretability, all of which are relevant to the clinical domain. Other studies evaluate these models on datasets that largely differ from MSK imaging, hence their conclusions cannot be equated to the challenges of MURA. There are very few comparisons that use the MURA dataset and even fewer that investigate how architectural differences influence behaviour under identical training pipelines. This dissertation address that gap by utilising architecture as the primary variable under controlled conditions.

2.9 Architectural Philosophy Debate

A persistent debate in deep learning concerns whether architectural depth or parameter efficiency is more valuable when designing a model. Deep models like ResNet have increased representational capacity that enables better feature extraction in complex tasks, particularly where subtle visual features must be learned from less data. Compound scaled architectures like EfficientNet propose that parameter efficiency achieves superior or at least competitive performance while lowering computational cost, making these types of models more practical

to deploy into a real world setting. Neither design philosophy is supported by unanimous evidence, performance advantages differ according to dataset properties, training conditions and evaluation criteria. This debate directly motivates the controlled comparison undertaken in this dissertation.

2.10 Evaluation Challenges in Medical Imaging

Another divide concerns how model performance should be measured. Many studies are centred around accuracy, reporting on overall accuracy or AUC. However, in MURA the task is to identify abnormal studies. Because abnormal cases are typically less frequent and more consequential (Bontempo, et al., 2025), several authors argue in favour of evaluation across multiple metrics. This is to primarily avoid models appearing strong while underperforming on minority classes. This nuance is particularly important for MURA, where false negatives carry a greater risk and consequence than false positives.

The Stanford MURA competition (Rajpurkar, et al., 2018) further demonstrates this. The competition challenged participants to develop models capable of detecting abnormalities at a level comparable to professionals in the field, using the same study labels for reference. Although the leaderboard demonstrated strong contenders, even the best did not rival the professionals' accuracy (Rajpurkar, et al., 2018). This might be because of the complexity MSK diagnosis presents, seven entirely different body parts are combined with the subtle nature of abnormalities. These factors are challenging for models trained on image features without the ability to reason like a clinical expert. This outcome reinforces the need to evaluate frameworks that go past overall accuracy as well as consider class imbalances, study level aggregations, etc.

Given the challenges above, MSK abnormality detection should prioritise metrics that reflect the clinical consequence of classification mistakes rather than accuracy alone. Specifically, abnormal class recall at the study level is the most important metric in this study because it directly measures the model's ability to detect abnormal studies while accounting for the cost of false negatives. Precision and F1 score provide more insight into the precision to recall tradeoff and the study level aggregation at the image level ensures evaluations lie in accordance with the clinical reality of diagnosis being made at the patient study level rather than on individual images (The Royal College of Radiologists, 2018).

False negatives, which in this context refer to where an abnormal class is incorrectly classified as normal may often lead to missed diagnoses and delayed treatment (Burt, et al., 2017). For

this reason abnormal recall is treated as the primary evaluation metric in this comparative study.

2.11 Interpretability and Evaluation Priorities

A recurring challenge in deep learning; especially for medical imaging is the limited ability to interpret CNNs (Teng, et al., 2022). Although CNNs achieve strong performance in image recognition tasks, including medical diagnosis. However, their internal representations are often hazy, making it difficult to understand why exactly certain image regions influence a prediction. This issue is answered in part by the phenomenon known as 'shortcut learning', by which a model will classify based on seemingly irrelevant information (Geirhos, et al., 2020). This lack of transparency is widely considered as the reason why AI is second guessed before being adopted into the medical world (Muhammad & Bendechache, 2024).

To address this, Gradient weighted Class Activation Mapping(Grad-CAM) has become a widely adopted interpretability method for CNN analysis (Selvaraju, et al., 2017). Grad-CAM generates localisation maps by computing the gradient of the predicted class score with the features maps of the final convolutional later, this produces a heatmap that highlights which regions influence a model's decision the most. Unlike earlier visualisation method such as vanilla backpropagation, Grad-CAM does not discriminate by class and does not require any architectural modification, making it applicable to any CNN without training again.
In medical imaging Grad-CAM has been applied to assess whether models attend to relevant regions or rely on spurious correlations or other artefacts. (Diaz, et al., 2026) demonstrates its use in detecting bias in binary medical classification, (Muhammad & Bendechache, 2024) highlight its role in trusting AI via providing transparent explanations of model decisions. In MSK context, Grad-CAM can help determine whether a model is responding to bone structure abnormalities or other contextual cues like medical supports, markers or background intensities. This is important given that all images in MURA are grayscale, it also removes the colour based shortcuts that are available in coloured image classification.

The application of Grad-CAM to both models in this dissertation allows for a qualitative comparison of how each architectural design influences model attention. The MURA challenge highlights the difficulty of achieving good accuracy scores on abnormal studies, further reinforcing the need for accordingly assessed metrics and interpretability. In medical imaging tasks, evaluation metrics should reflect the real world clinical consequences of classification errors.

2.12 Background Summary

The literature reviewed in this chapter shows that CNNs have become a dominant approach for image based tasks, including applications within medical imaging. ResNet50 and EfficientNet-B0 represent well benchmarked and strong performances in the medical domain. One prioritises depth and residual learning and the other focuses on parameter efficiency through compound scaling. Research studies indicate that both architectures are capable of strong performance, relative advantages differ depending on dataset, evaluation metrics and computational constraints.

However, existing research has many limitations. Most studies focus on overall accuracy, without exploring training behaviours, study level aggregation or interpretability. A large number of research papers are conducted on datasets that are significantly different from musculoskeletal radiography. The few that use MURA remain limited, and there is little investigation into how architectural design influence model behaviour when training conditions are controlled and identical.

Given these limitations, there remains a need for a structured comparison that isolates architectural impacts while also considering evaluation practices that better reflect real world clinical practice. This dissertation attempts to address that gap by comparing ResNet50 and EfficientNet-B0 under identical training pipelines using MURA.

3 Methodology

3.1 Experimental Approach

This project adapts a methodology based around a controlled comparison of two CNN architectures; ResNet50 and EfficientNet-B0. The main objective is to evaluate how architectural differences influence performance, generalisation and robustness when models are trained under identical conditions using transfer learning.

To achieve this, I created an identical training and evaluation pipeline to use for both models, allowing them both to be trained and assessed using the same dataset and the same preprocessing steps, optimisation strategy and evaluation metrics. This ensure that differences in behaviour are a result of each models individual architectures rather than experimental nuances.

The design focuses on a controlled comparison between the 2 models, pretrained using transfer learning and then trained on the MURA dataset. Data augmentation is applied during training to improve model robustness to variations that occur in image orientation and appearance. Model performance is evaluated using standard metrics like precision, recall, F1 score across weighted and macro averages. Experiments are conducted across different dataset splits and fixed random seeds to examine stability and model generalisation. This approach allows for a deeper evaluation.

The design intentionally focuses on dataset partitioning strategies rather than fine tuning hyperparameters repeatedly. Hyperparameter optimisation can often dilute underlying model behaviour by introducing additional means of variation (Schneider, et al., 2025).

3.2 Dataset and Dataset Exploration

This project uses the Musculoskeletal Radiographs (MURA) dataset provided by Stanford University. The dataset contains radiographic studies labelled as normal or abnormal at the study level. Each of the studies consists of a couple associated images. In this dataset, labels are assigned at the study level, however model training and evaluation are done at the image level. The dataset is divided into 2 subsets; training and validation using the split provided by the designers of MURA. This split is done at the study level to ensure all images that belong to a given study are assigned to a training or validation set. This prevents information leakage between the subsets. For example, no images from the validation study are seen during training. Model learning is therefore restricted to only the training set, and model performance as a whole is only evaluated on unseen validation data.

However, before implementing the main training pipeline, an exploratory Python script was developed to understand the structure and organisation of the MURA dataset. The step was done to verify how studies and images were arranged, and to understand the relationship between study level labels and singular images. After this it was clear that labels are assigned at the study level, each of which contained a couple images associated to that study and body part. This was an important step because it informed the design of the dataset image loading pipeline. Conducting this analysis reduced the risk of misinterpreting the structure of the dataset and strengthened the design of the experimental setups.

Minimal dataset preprocessing was done prior to training. Image files were filtered based on valid file extensions to remove non-image and or corrupted files that do not have a meaningful contribution to the learning process. This step was only implemented to ensure reliability when

loading data and so that it does not alter the content or distribution of the dataset. No extra manual filtering, alteration of labels or dataset balancing was applied. This allowed models to be trained on data that reflects the natural design and behaviours of the MURA dataset.

3.3 Dataset Partitioning and Strategy

The MURA dataset consists of 40,561 images. In the publicly released partitioning, approximately 36,800 images are used for training and 3200 for validation, representing a rough 91/9 split. There is no publicly available test split in the official release. Partitioning medical datasets requires a balance of training stability with reliable evaluation. Datasets like MURA are relatively small and exhibit a potential for class imbalances. Many studies adopt training heavy splits to maximise the learning signal carried through to deeper architectures.

The original MURA authors used a 90/8/2 split (Rajpurkar, et al., 2018), which has been widely replicated by studies to remain comparative with the benchmark. Performance estimates can become unstable when evaluation sets are too small, and reducing the training set can impair generalisation — this is known as the stability-accuracy trade off (Singh, et al., 2021). A further challenge arises from the stochasticity of deep learning methods. Random weight initialisation, order of shuffled data and augmentation variations heavily influence training trajectories, potentially leading to misleading results across runs. Literature recommends using multiple seeds (Wegmeth, et al., 2023) to reduce this variance and improve understanding of model behaviour.

To ensure an unbiased and reliable evaluation of model performance (Babaei, et al., 2025), a structured dataset split strategy was implemented. The dataset was not divided at the image level but rather by patient. This was done to prevent data leakage between the training, validation and testing subsets. Initial experiments were conducted without a dedicated test set in order to benchmark model behaviour during development. These results are not part of any formal evaluations. Main analysis and evaluation were conducted using an 80/10/10 split, which is widely used in machine learning since it prioritises the amount of data in training while maintaining a fair amount for validation and testing. To assess robustness a 70/15/15 split was added to allow for comparison against how models react with less training data. To allow the results to be reproducible, dataset partitioning was controlled using fixed random seeds to eliminate the impact of random shuffling. This makes sure that results from experiments remain consistent across runs to allow for fair comparison between architectures. In total, 3 fixed random seeds were implemented to create a larger more interpretable base for results and evaluations to be drawn from.

In medical imaging datasets like MURA, multiple images often belong to the same study and patient. CNNs take in individual images, so if an image from the same patient were spread across different subsets, it would be unavoidable for the model to learn patient specific traits rather than generalize based on the diagnostic features of the image. To prevent this, dataset partitioning was performed at the patient level so that all images from a specific patient always remained within the same subset.

The original MURA dataset provides separate training and validation lists but does not include a dedicated testing set. To support an unbiased investigation, studies from the original training and validation sets were first combined into a single large pool from which new splits were generated. Patient identifiers extracted from study paths were used to make sure that each patient was only assigned to a single subset, this design allows the final test set to be completely unseen by the model during training and validation.

The test set was strictly isolated and not assessed during training or validation at any point. This ensures that all results reflect true performance.

3.4 Model Selection and Transfer Learning

Both models were implemented using PyTorch's deep learning framework, loaded from the 'torchvision.models' library. To make use of existing visual features, both networks were initialised with weights pretrained on the ImageNet dataset (Deng, et al., 2009). It was selected because it is one of the largest and most diverse image classification datasets available. Features learned from ImageNet have been shown to apply effectively to medical imaging domains, providing models with a strong ability to capture features and reduce the among of data needed to learn task relevant data in this medical context (Raghu, et al., 2019).

Small modifications were made to adapt the models for binary classification. For both models, the original classification layer was replaced with a new output layer which is needed to distinguish between normal and abnormal cases. Apart from that minor adjustment, the main architecture of each model remains the same. This ensures that the comparison is focused on each model's architectural differences rather than custom design alterations.
Transfer learning was applied by fine-tuning both models on the MURA dataset. After replacing the classification layer, the network weights were updated during training of the models, allowing the pretrained features to adapt to musculoskeletal radiographic data. A similar

strategy for transfer learning was used for both architectures to ensure a fair and controlled comparison.

3.5 Training Procedure

Both models were trained using a controlled training procedure to ensure a fair comparison, the procedures were consistent across both models. For the binary classification, a cross-entropy loss function was used. This is appropriate for supervised classification problems and widely used in image based recognition programs. Cross-Entropy (Goodfellow, et al., 2016) was applied to both models during training.

I used the Adam optimiser with a fixed learning rate of 1×10^{-4}. I chose the Adam optimiser (Kingma & Ba, 2015) because of its effectiveness and stability when fine tuning pretrained neural networks. A batch size of 16 was used for training in order to balance computational power efficiency properly with memory constraints, this is consistent with practice observed in medical imaging tasks in datasets of a similar scale (Mienye, et al., 2025). During final analysis each of the models was trained for a fixed number of 10 epochs. All training hyperparameters were kept the same across models to avoid the risk of bias during experiments.
At the end of each epoch, the model's validation loss was computed and compared to the lowest validation loss observed in that run. If validation loss improved, the model's weights were preserved and saved as a checkpoint, see Fig 2 for example.

```
Best model saved at: ..\checkpoints\resnet50_seed_21_best.pth
```

Figure 3.1: Checkpointing Confirmation in Training Script

Final testing for a model was carried out using these weights. This ensured that model's final report could be carried out on the best possible conditions. Models were trained for a set number of epochs to maintain consistency across experiments. In total this study contains 14 runs, 12 of which were the final runs used in analysis, Each model was trained a total of 6 times, using 1 of 3 seeds and 1 of 2 dataset splits across all possible combinations.

By using the same loss function , optimiser, learning rate, batch size for both models. The training process makes sure that differences in performance can be credited to model architecture as opposed to variations in training.

3.6 Data Preprocessing and Augmentation

All input images were resized to a resolution of 224 x 224 pixels to ensure consistency across models and compatibility with the pretrained architectures. ImageNet mean and standard

deviation was used to normalise images. This aligns the input distribution with what is expected by the pretrained networks. This also supports stable transfer learning.

Data augmentation was applied during training in an attempt to improve model generalisation as well as reduce overfitting (Shorten & Khoshgoftaar, 2019). For the training set, as afore mentioned images were resized to a fixed resolution, in addition to that the images also undergo random horizontal flips and colour jitter transformations. These augments are widely applied in medical imaging pipelines and have been shown to improve model generalisation (Veličković, et al., 2018)These augmentation create a controlled variance in the images, thus encouraging the models to learn more orthodox and groupable features. By using image normalisation via ImageNet's mean and standard deviation, inputs were better aligned with pretrained network expectations.

Pretrained representations have been shown to transfer effectively in medical settings (Raghu, et al., 2019)

More importantly augmentation was applied only to the training set of images. The validation set only had standard resizing and normalisation applied to it. This makes sure that validation performance purely reflects the model's ability to class unseen data as opposed to its abilities under altered conditions. Maintaining an evaluation pipeline with a sole purpose prevents manipulation of evaluation metrics and preserves the integrity and aim of model comparison.

3.7 Reproducibility Controls

To confirm that the random seed implementation was working correctly, each model was trained twice during test run using the same seed and identical settings. The results were the same across both models; shown in Figure 3.2 and 3.3. This shows that fixing the seed produces consistent outputs and that the training process is reproducible under controlled conditions. Metrics are rounded to 4 decimal places.

Table 3.1: Seed 15 Similarity for EfficientNet-B0

Metric	Run 1 – Seed 15	Run 2- Seed 15
Train loss	0.5074	0.5074
Validation loss	0.4742	0.4742
Image accuracy	0.7817	0.7817
Study accuracy	0.7932	0.7932

Table 3.2: Seed 15 Similarity for ResNet50

Metric	Run 1 – Seed 15	Run 2 – Seed 15
Train loss	0.5272	0.5272
Validation loss	0.5587	0.5587
Image accuracy	0.7604	0.7604
Study accuracy	0.7731	0.7731

The same augmentation pipeline was used for ResNet50 and EfficientNet-B0 to ensure consistency and fairness. Any differences observed in performance can be concluded as a result of a models architecture rather than discrepancies in data preprocessing

3.8 Evaluation Framework

As mentioned previously, model performance was evaluated using standard classification metrics like accuracy, precision, recall and F1 score computed at image and study level. Image level interpretations provide insight into singular images while study level predictions aggregate predictions across images belonging to the same study, essentially providing insight into diagnosis the same it would be concluded in a clinical setting; as a whole.
Model architecture was assessed using a separate test set, model performance was assessed on this test set using model weights from the best training epoch, dictated by lowest validation loss.

The selection of precision, recall and F1 score as the metrics used in this study lays in line with best practice for binary classification especially in medical imaging, where accuracy alone is considered inefficient (Hicks, et al., 2022).
Particular emphases was placed on abnormal recall. In medical imaging tasks, failing to detect an abnormality i.e. a false negative can carry more severe consequences than incorrectly classifying a normal case as abnormal i.e. a false positive. This emphasis lays in accordance with the clinical importance of minimising missed diagnoses, this is seen in literature (Bontempo, et al., 2025).

No ranking or scoring scheme was used in this dissertation to allow for a transparent and interpretable comparison of model behaviour.

Grad-CAM was chosen over other visualisation techniques such as LIME or SHAP, Grad-CAM does not require architectural modifications and is widely validation in the medical field (Selvaraju, et al., 2017). By generating heatmaps that highlight relevant regions for each model's final prediction, Grad-CAM will allow assessment of whether each architecture attends the meaningful structures. This is imperative in a comparative study since the objective is to understand reasoning behind conclusions rather than their accuracy, allowing this dissertation to provide a more complete explanation of architectural behaviour.

3.9 Ethical Considerations

This project was classed as an NER(no ethics required) because it involves secondary analysis of an existing dataset containing anonymised images. No participants were required. The dataset was used in accordance with its research usage terms, all experiments were carried out for non-commercial purposes. None of the data collected was or ever will be deployed in a real world clinical setting. Therefore, the project represents minimal ethical risk, and obtained ethics approval from Brunel University prior to any experimentation. Proof available in Appendix B.1.

4 Design

4.1 Experimental System Architecture

The experimental framework for this dissertation was designed so that both CNN architectures could be trained and evaluated under the same conditions. Establishing this identical pipeline is important in controlled model comparisons (Bouthillier, et al., 2021). It also ensures that any differences in performance are a result of architectural design rather than experimental variations. The system architecture used for data processing, model training and results evaluation is illustrated in Figure 4.1.

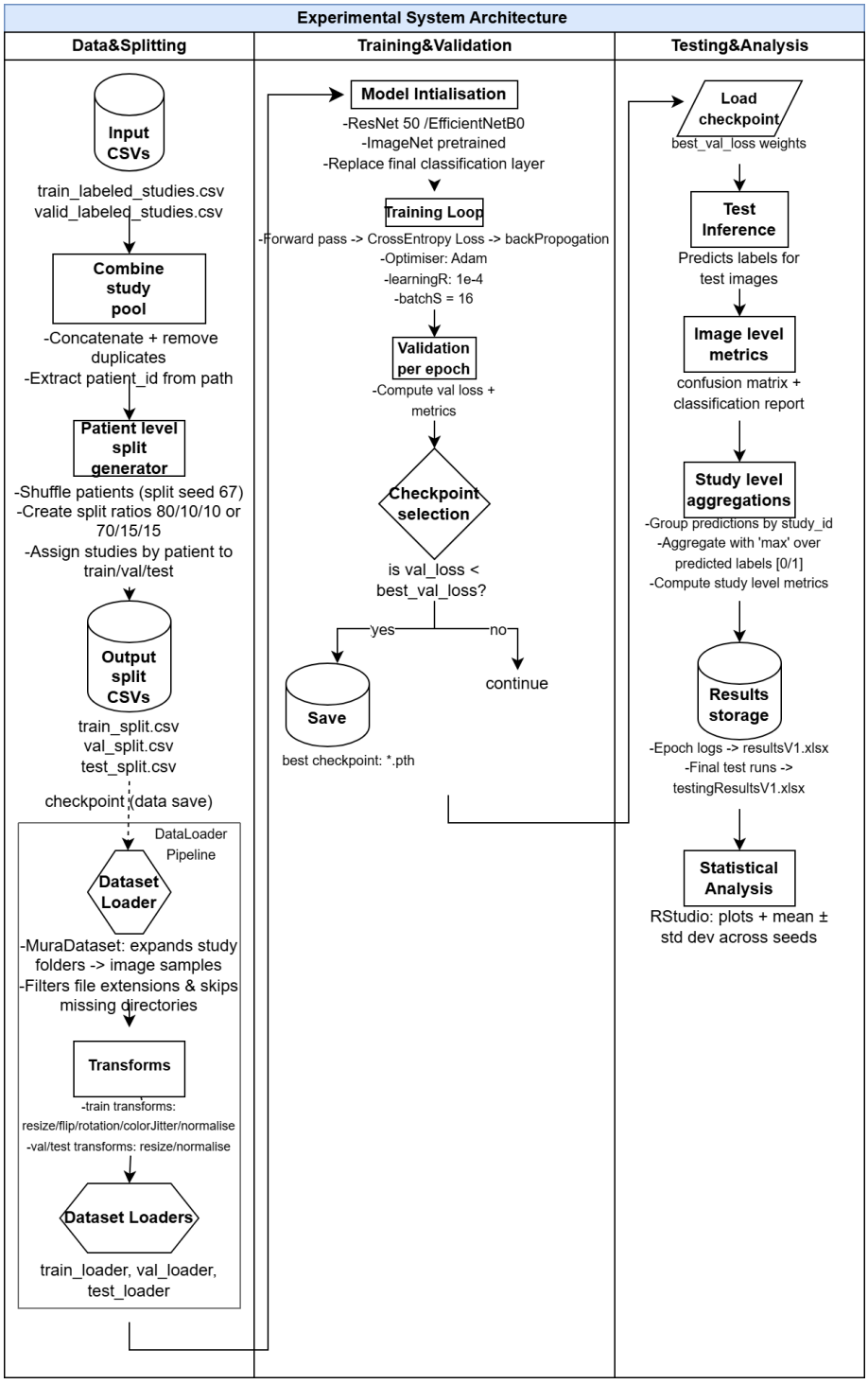


Figure 4.1: Experimental System Architecture

The Figure above shows that the pipeline begins with the MURA dataset and metadata files. The mentioned training and validation files are combined to form a complete pool of available studies. From this pool, patient identifiers are extracted and used to generate patient level dataset partitions using 1 of 3 fixed split seeds. This ensures that every image that belongs to the same patient remains within a single subset, helping to prevent data leakage between training, validation and testing sets.

After this dataset partitioning, the image loading pipeline expands each study directory into individual images. Data preprocessing and augmentations are then applied in the transformation pipeline. These augmentations are only applied to the training set since the main focus of validation is to evaluate the model's actual ability to generalise on real unaltered images. The processed images are then delivered to the training process through PyTorch's 'DataLoader' function.

The training stage consists of forward propagation through the CNN model, computation of the cross entropy loss function and parameter updates using the Adam optimiser.
During each epoch, validation loss is monitored and compared against the previously observed value. When an improvement in validation loss occurs, the model weights are stored as a checkpoint representing the best performing state during training.

The choice to use 10 epochs was informed by observations from the pilot runs that indicated that validation loss stabilised and began to diverge from training loss within this range for both models, suggesting that additional epochs may have contributed to overfitting rather than improving model generalisation. This is consistent with early stopping where training duration is guided by validation performance rather than a fixed point (Prechelt, 1998).

After training is completed after all 10 epochs, the same 'best checkpoint' is loaded and used on the held out test dataset. Image level predictions produced by the model are then grouped by study identifiers and aggregated to produce study level predictions. Finally, evaluation metrics are calculated and exported to spreadsheet files for further statistical analysis and visualisation.

4.2 Experimental Configuration Design

To evaluate the behaviour of the 2 architectures under controlled conditions, a structured experimental configuration was used. The design incorporates variations in dataset partitioning and random initialisation while maintaining consistent training hyperparameters; batch size 16, learning rate 1e-4 and 10 epochs across all runs. The configuration matrices for ResNet50 shown in Figure 4.2 and EfficientNet-B0 in Figure 4.3 respectively.

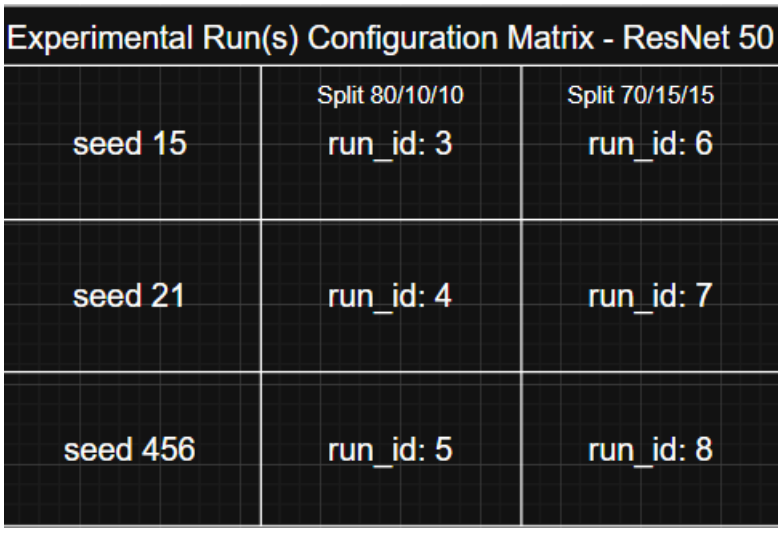


Figure 4.2

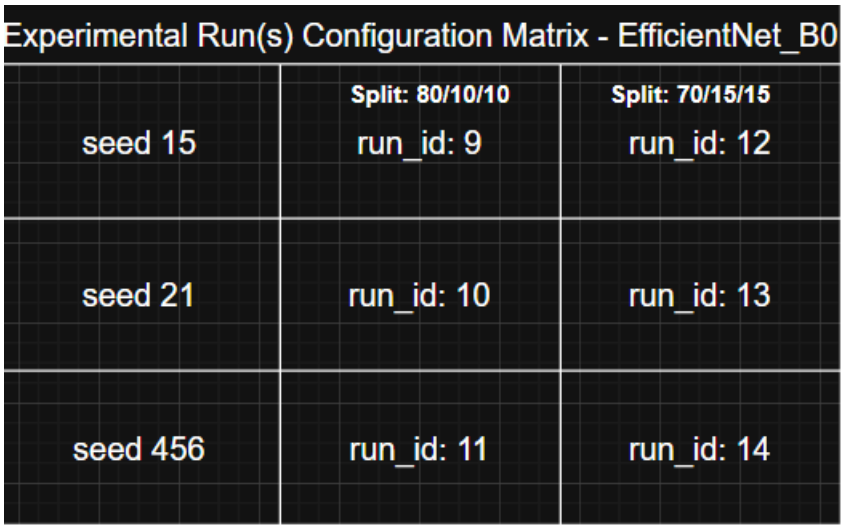


Figure 4.3

As shown in the Figures above the experiments are organised by 3 independent variables; model architecture, dataset splits and seed initialisation. Two architectures are evaluated in this study, ResNet50 and EfficientNet-B0. For each of the architectures, experiments are conducted using an 80/10/10 split and 70/15/15 split for training, validation and testing respectively. The experimental configuration was designed to evaluate behaviour under controlled stochastic conditions. Training deep neural networks often involves several random processes like weight initialisation, dataset shuffles and mini batch sampling which can dictate optimisation and results trajectories. Evaluation models using a single training run may produce results that are not representative of an architecture's true behaviour. To address this issue, each configuration was carried out across multiple fixed seeds. This enables experiments to capture variations adequately and gives a more reliable base for comparing architectural stability.

To account for stochastic variations during CNN training, three fixed random seeds are used across experiments. Each seed produces a separate training run while preserving deterministic behaviour and allowing randomness to be controlled and reproduced in weight initialisation. A separate fixed seed (seed 67) was applied to the dataset to prevent random dataset shuffling. This approach enables the evaluation of model stability and ensures that conclusions are not drawn from a single random initialisation.

All experiments were executed using the aforementioned training parameters (batch size, learning rate, epochs). Maintaining these controlled variables ensures that differences can be attributed to a model's architecture rather than variations in training configurations.

4.3 Study Level Prediction Aggregation

Although model training and interpretations are performed at the image level. MURA provides labels at the study level. Because of this a mechanism is required to convert the image level predictions into a single study level classification. The study aggregation used in this project is illustrated in Figure 4.4.

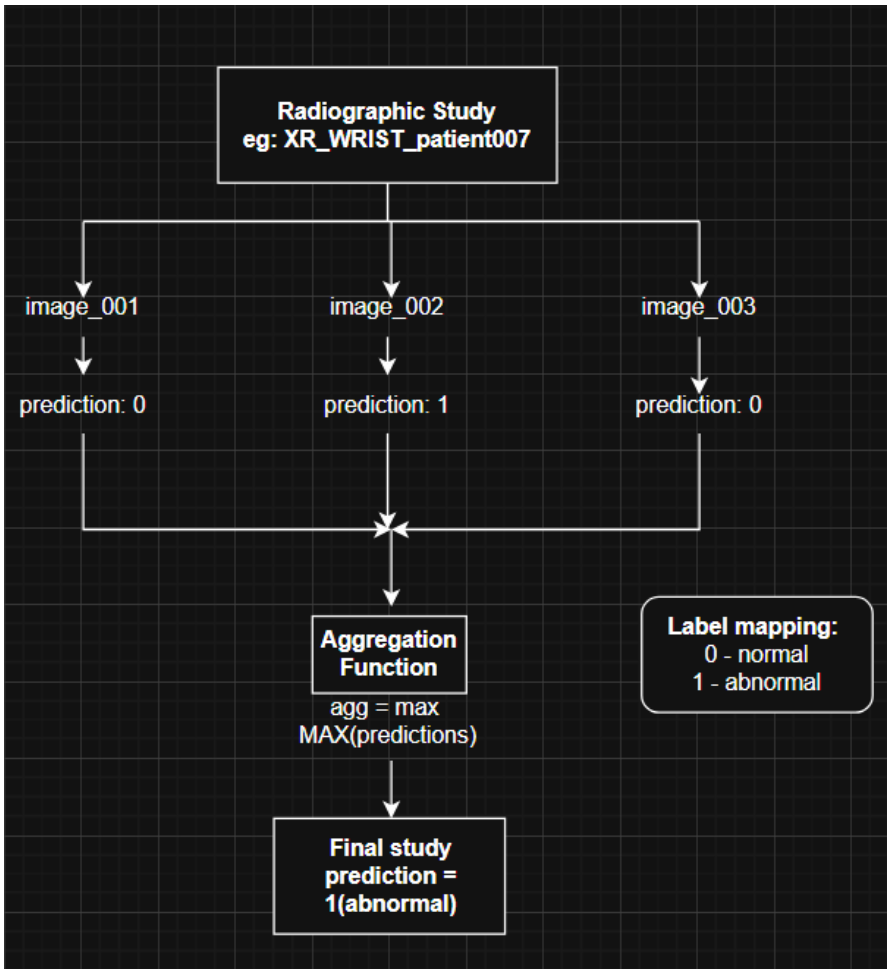


Figure 4.4: Study Level Prediction Aggregation

As shown above, each study may contain multiple associated images. During evaluation, each image is independently processed by the trained CNN model to produce image level predictions, classing an image as normal or abnormal.

To derive a study level classification, image predictions belonging to the same study are grouped and aggregated using a maximum aggregation rule. If any image within a given study is predicted as abnormal, the whole study is then classified as abnormal. This aggregation choice reflects a conservative diagnostic outlook, where the presence of an abnormal finding in any image should result in should result in the entire study being flagged as abnormal (The Royal College of Radiologists, 2018).
The resulting study level predictions are then used to compute evaluation metrics that reflect the model's performance from a clinical study perspective rather than only at the image level.

4.4 EfficientNet-B0 and ResNet50 Model Architectures

ResNet50 is centred around residual learning, skip connections allow gradients to flow better through this deep architecture. This design helps in mitigating the vanishing gradient problem (He, et al., 2015) and enables stable training patterns. Due to this design, ResNet models are often associated with more consistent optimisation behaviour and strong baseline performance across image classification tasks. Figure 4.5 demonstrates how ResNet models differ in their design as compared to most CNNs. ResNet50 is essentially an extended, 50 layer version of the architectural design labelled as 'ResNet' below.

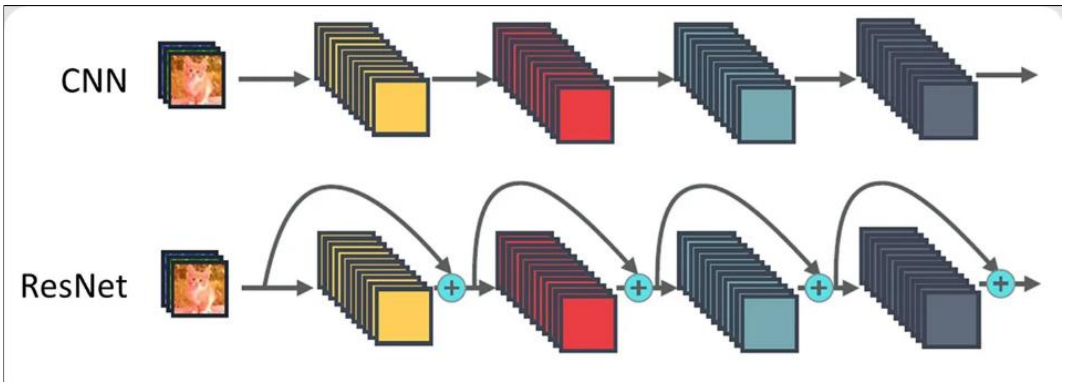


Figure 4.5: Simple Design Concept of ResNet Models vs Traditional CNNs (Vina, 2025)

On the other hand, EfficientNet-B0 is part of a family of models that make use of compound scaling, this means that network depth, width and resolution are scaled in a balanced manner. This approach to CNNs aims to achieve higher performance while maintaining parameter efficiency, meaning that computational resources are used more efficiently, like the name suggests. EfficientNet architectures, as seen in Figure 4.6 are designed to extract more

meaningful features using fewer parameters, leading to improved computational efficiency as compared to traditional CNN designs.

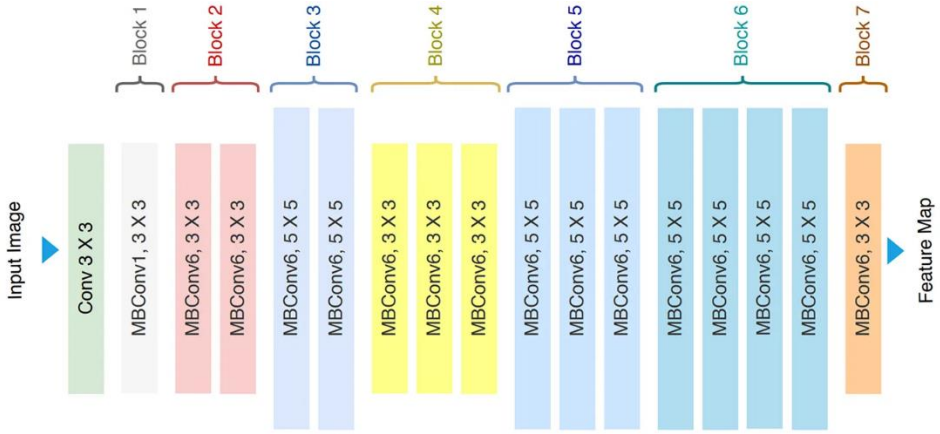


Figure 4.6: Design Concept for EfficientNet-B0 (Ahmed & Sabab, 2022)

This model is organised into a sequence of blocks where each stage processes the image and extracts increasingly complex information. The model is built using MBConv blocks which are designed to process information by focusing on more meaningful features. As mentioned, depth, width and input resolution are increased together rather than independently allowing these type of models to perform better by accounting for the mathematical and structural interdependencies between network dimensions while remaining efficient.

Understanding the structure of these 2 models is important for this dissertation because it influences how effectively models extract meaningful features that contribute to accurate classifications.

4.5 Grad-CAM Design

Grad-CAM was added into the experimental design as a means of qualitative interpretation to compliment the quantitative evaluation. This design choice to apply Grad-CAM to the final convolution layer specifically was made deliberately because the final layer contains the highest level of information while preserving spatial resolution for meaningful heatmap creation. For ResNet50, the target later is the final residual block. For EfficientNet-B0, the target layer is the last MBConv block before the classifying layer.

A forward hook and backward hook are attached to the target layer, the forward hook captures feature maps created during the forward pass while the backward hook captures gradients flowing back through that same layer during backpropagation. Global average pooling is then applied to the captured gradients to create channel wise weights, which are then used to compute a weighted combination of the feature maps. A ReLU activation is applied to retain

only positive contributions, the final map is up sampled to match original image dimensions using bilinear interpolation.

This design was applied to both architectures to ensure comparisons are equal. Heatmaps were generated on validation set images for correct abnormal predictions, current normal predictions and a misclassification case for each. This provides a structured basis for qualitative comparisons of architectural behaviour.

5 Implementation

5.1 System Architecture and Code Structure

The experimental system described in the Design Section was implemented as a modular Python project, allowing separation of core components of the training pipeline and simplify experimental configuration for easy and ease of use. The Python file structure is shown in Figure 5.1 in the Visual Studio Code IDE. The Figure shows the main modules used to implement individual workflows (training, model selection, transforms, etc). Organisation of the codebase in this manner allowed individual components to be changed without affecting the whole pipeline. Please note that that 2 'inspect_mura' files are not part of the final implementation and are omitted from the system design.

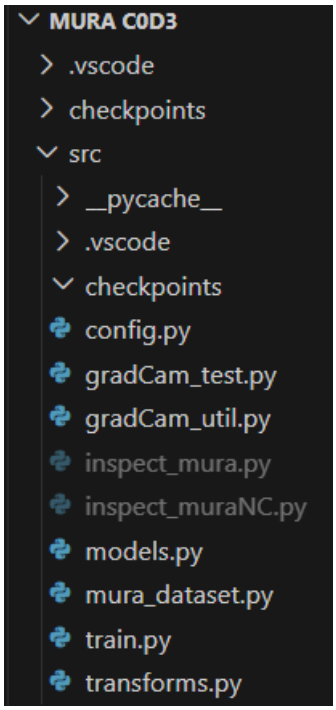


Figure 5.1: File Directories in VSCode

The main training logic exists in 'train.py', responsible for the entire experimental process, this includes dataset loading, model initialisation, training, validation, evaluation and checkpoint management. Supporting files like 'config.py' exist to house specific experiment settings and default values. Model architectures are available in the 'models.py' file, where the pretrained CNNs from the torchvision library and initialised. This is also where the final layer of both models was modified for binary classification as required for this task.

Dataset handling is implemented through the custom MuraDataset class in 'mura_dataset.py'. This file manages image loading, individual label extraction and the association between images and their corresponding studies. Image preprocessing and augmentation operations are carried

out separately in ‘transforms.py’, this allows data preparation is modular and reusable ate training and evaluation stages.

The qualitative interpretability of this dissertation is implemented through 2 files, ‘gradCam_util.py’ contains the main Grad-CAM implementation responsible for extracting feature map gradients and generating heatmaps. The ‘gradCam_test.py’ file uses the utilities in the previously mentioned file to apply Grad-CAM to the trained models to produce visualisations as seen in Section 6.8.

This structure allows the project to remain organised and maintainable, allowing components to be singled out and modified as required for different stages of this project.

5.2 Dataset Loading and Preprocessing Implementation

The MURA dataset is organised at the study level, each directory contains a couple of associated images, but the whole directory is organised with a single label. Since CNNs operate on individual images, a custom dataset loader was implemented to convert the study level structure into image level training samples while keeping track of the relationship between image and study.

The dataset loading is done through the custom ‘MuraDataset’ class which extends the prebuilt PyTorch Dataset interface. The dataset loader reads the study paths and assigned labels from the associated MURA CSV files, it then expands each study into individual images. For each study, the loader creates an absolute directory path, verifying that the directory exists. This also helps keep track of which image originates from which study and scans the directory to verify where image files originate from. Only files recognised with valid image extensions are included in the dataset to avoid the possibility of processing non image files that can crash the whole system.

As shown in Figure 5.2.1, the loader iterates through each study and generates a list of individual samples. Each of these contains the image path, study label and generated study_id. The identifier is taken from the study path and used later in the pipeline to aggregate image level predictions into study level predictions in evaluation.

```
self.df = pd.read_csv(csv_path, header = None) #tell pandas csv file do
                                                #treats first row as dat
self.df.columns = ["study_path", "label"] #naming rows instead of auto

study folder into single images
self.samples = [] #empty list to collect image path, labels, and study
for _, row in self.df.iterrows(): #looping over each csv row
    study_path = row["study_path"]
    label = int(row["label"])

ath, os format issues
    study_path = study_path.replace("\\", "/")
    prefix = "MURA-v1.1/" #strips MURA from path, bcuz alr exists in
    if study_path.startswith(prefix):
        study_path = study_path[len(prefix):]
    study_dir = os.path.join(self.root_dir, study_path)

ages in directory
    if not os.path.isdir(study_dir): #builds full folder path
        continue #skip listing if image missimg

    valid_exts = (".png", ".jpeg", ".jpg")
    image_files = [ #look in study folder, only take files ending in
                    #this becomes the list of images we can train on
        f for f in os.listdir(study_dir)
        if f.lower().endswith(valid_exts) and not f.startswith(".")
    ]
    for img_file in image_files: #for every image
        #study id can b folder name, i.e. study1+ve
        img_path = os.path.join(study_dir, img_file)
        study_id = study_path.replace("/", "_") #study id = study f
        self.samples.append((img_path, label, study_id))
```

Figure 5.2.1: Dataset Sample Construction

Once samples are created, the dataset class uses the '__getitem__' method to return a singular training example. During sample retrieval, each image is loaded from the disk, converted to RGB and passed through the transformation pipeline before being returned. In addition to the, the dataset loader also returns the associated study_id and image path. Keeping track of the study identifier is important because the evaluation requires predictions from multiple images belonging to the same study in order to be aggregated and finally outputted as a study level prediction.

```
def __len__(self): #how many samples
    return len(self.samples)#telling pytorch the
                            #pytorch must know t

def __getitem__(self, idx): #return 1 sample
    img_path, label, study_id = self.samples[idx]

#load image in rgb
    image =  Image.open(img_path).convert("RGB")

    if self.transforms: #if we pass augmentations
        image = self.transforms(image)

    return { #return a dictionary with the follow
        "image": image,
        "label": label,
        "study_id": study_id,
        "image_path": img_path
    }
```

Figure 5.2.2: Retrieval Logic

Image preprocessing and augmentation is implemented separately the transforms pipeline found in 'transforms.py'. This separation is important and allows preprocessing operations to be reused across different stages of the pipeline while allowing the dataset loader to focus on data retrieval. 2 transformation pipelines exist, one for training and one for validation/testing. The training pipeline includes resizing, flipping, rotation and jitter to introduce variation during training in the attempt to improve model generalisation. The other pipeline only applies resizing and normalisation to ensure consistent evaluation.

```
def get_train_transforms(img_size):
    return Tr.Compose([
        Tr.Resize((img_size, img_size)), #resizes t
        Tr.RandomHorizontalFlip(p = 0.5), #randomly
        Tr.RandomRotation(degrees = 10), #slight ra
        Tr.ColorJitter(brightness = 0.1, contrast =
        Tr.ToTensor(), #converts PIL to pyT tensor
        Tr.Normalize(mean = [0.485, 0.456, 0.406],
                     std = [0.229, 0.224, 0.225]),
    ])

#validation transforms
def get_valid_transforms(img_size):
    return Tr.Compose([
        Tr.Resize((img_size, img_size)),
        Tr.ToTensor(),
        Tr.Normalize(mean = [0.485, 0.456, 0.406],
                     std = [0.229, 0.224, 0.225]),
    ])
```

Figure 5.2.3: Preprocessing Pipelines

The separation in dataset loading and preprocessing while retaining study identifiers allows the training pipeline to work on individual pipelines while allowing for study level evaluation later.

5.3 Study Level Prediction Aggregation

To account for MURA's structure that include study level labels, the evaluation pipeline requires a mechanism to convert image level predictions into a singular study level prediction. This aggregation process was implemented directly within the training script alongside the evaluation stage.

As shown in Figure 5.3, predictions are first grouped according to their corresponding study_id. A dictionary is used to collect predictions that belong to the same study, each entry represents a unique study identifier and the associated values storing a list of image level predictions. At the same time, a separate dictionary is in charge of storing the truth label for each study.

```
study_preds = defaultdict(list) #initializes a dictionary to
study_labels = {} #stores single truth label per study

for pid, pred, label in zip(study_ids, predictions, labels):
    # normalize study id to a string so it always hashable a
    pid_key = str(pid)
    try:
        study_preds[pid_key].append(int(pred))
    except Exception:
        study_preds[pid_key].append(pred)
    study_labels[pid_key] = int(label)
#loops thru each prediction n groups by study id

final_preds = []
final_labels = []
#lists to store aggregated study predictions n ground truths

for pid, p_list in study_preds.items():
    if agg == "max":
        final_pred = max(p_list) #study predicted abnormal i
    elif agg == "mean":
        final_pred = sum(p_list) / len(p_list)#avg the predi
    else:
        raise ValueError("Unknon aggregation method")#throw

    # ensure prediction is int 0/1 abnormal or normal
    try:
        agg_pred = int(round(final_pred))
    except Exception:
        agg_pred = final_pred

    final_preds.append(agg_pred) #adds the rounded agg predi
    final_labels.append(study_labels[pid])
```

Figure 5.3: Study Level Aggregation using Max Aggregation

Once all predictions are grouped, the aggregation function is applied. In each case, the list of image level predictions is calculated into a single study prediction using a maximum aggregation rule. This is seen in the 'max( )' function that selects the highest predicted value among every individual image prediction. The same predicted value is then converted into a binary output representing either a normal or abnormal study.

This method ensure that a study is classified as abnormal so long as an image within that study produces a strong abnormal prediction. This prediction is then added to the final list of study level predictions with its study label.

By doing this, this experimental framework is able to work around the mismatch between the way a CNN works with individual images and MURA's study level labelling structure.

5.4 Model Initialisation and Transfer Learning

The CNNs used in this dissertation were initialised using pretrained architectures from the PyTorch 'torchvision library' (Paszke, et al., 2019). By doing this, the CNNs were able to start their task from features learned from' ImageNet', which is a large image dataset. This significantly assists training efficiency and convergence as compared to training a CNN from scratch.

The implementation supports the use of a variety of models using the interchangeable parameter which specifies the exact model to be used. When a model is selected, the corresponding pretrained architecture is loaded using 'torchvision.models'. This is show in Figure 5.4, where both models are initialised using pretrained weights, allowing for already learned feature representation from the ImageNet dataset to be applied to this task.

```
if model_name == "resnet50":#load pretrained resnet50
    #weights=ResNet50_Weights.DEFAULT` to get the most up-to-d
     #precaution message from terminal^^^
    model = models.resnet50(pretrained = True)
    #replace final fully connected layer
    #resnet50 has fc as final layer
    in_features = model.fc.in_features
    model.fc = nn.Linear(in_features, num_classes)

elif model_name == "efficientnet_b0":
    #load pretrained efficienet b0
    #using torch vision implementation
    model = models.efficientnet_b0(pretrained = True)
    #replace classifier head
    in_features = model.classifier[1].in_features
    model.classifier[1] = nn.Linear(in_features, num_classes)
```

Figure 5.4: Initialisation of Pretrained CNNs and Alteration of Final Classification Layer

Albeit, both models are widely used for classification tasks, the task at hand involving the MURA dataset requires binary classification, because of this the final classification layer of each model was replaced to produce outputs in line with the 2 classes required for this task; abnormal and normal.

For ResNet50, this included replacing the fully connected 'fc' layer with a linear layer whose input features match the output dimensions of the prior network layers. For EfficientNet-B0 the final classification layer was replaced with a similar linear layer configured for binary classification.

By adapting the final layers while retaining each model's architectural integrity, this implementation enables transfer learning to be applied effectively to this task.

5.5 Training Pipeline and GPU Execution

In the main training script, the model parameters are updated using mini batch gradient descents. During each iteration of training, a set batch of images and corresponding labels are retrieved from the data loader and transferred to a selected device. This device is dynamically selected by detecting whether a CUDA enabled GPU is available for use, enabling the training process to make use of GPU acceleration when supported by the system. During this experimental implementation, the GPU used is Nvidia GeForce RTX4060.

As seen in Figure 5.5, the training loops follows a standard order of operations used in deep learning optimisation

```
images = batch["image"].to(device)
labels = batch["label"].to(device)

#standard training steps
optimizer.zero_grad() #clear prev
outputs = model(images) #feed imag
loss = criterion(outputs, labels)
loss.backward() #backprop gradient
optimizer.step() #upd8 model param
```

Figure 5.5: Training Loop showing GPU Execution and Optimisation

First, the image and label tensors are transferred to the selected device to ensure that both model and accessed data reside in the same area of memory. This is an essential step when utilising GPU acceleration, given that tensor related operations must be executed on the same device where model parameters exist.

Once that batch has been prepared, optimiser gradients are reset; 'optimizer.zero_grad( )', this is done to prevent gradient values from accumulating from previous iterations. This may lead to inaccurate weight updates, exploding gradients (Pascanu, et al., 2013). The model then performs a forward pass in which the batch of input images is processed by the CNN to produce predictions. After which, these predictions are compared against their true labels using a loss function, calculating the discrepancy between predicted outputs and expected labels.
After this loss computation, backpropagation is done using 'loss.backward( )'. This operation computes the gradients of loss, taking into account model parameters. It propagates the error signals backwards through the network's layers. The optimiser then updates the model parameters using these gradients via the 'optimizer.step( )' operation. Weights are adjusted to reduce loss incurred during following iterations.

By using CUDA, implementation significantly reduces the computation of forward and backward passes as compared to if training were carried out on the CPU. This allows training to be carried out within a reasonable timeframe.

5.6 Experimental Configuration and Reproducibility

To support controlled experimentation, the training pipeline was designed to allow key parameters to be specified externally by the user through command line arguments, which are also in charge of running the program. This approach enables the different configurations of the

experiment to be implemented without altering the original training script. This simplifies the process of running multiple experiments across different models, splits and seeds.

As seen in Figure 5.6.1, the implementation uses Python's 'argparse' module to define a set of parameters that are provided when launching the training process from the command line. These parameters include, model name, dataset paths, learning rate, batch size, number of epochs, dataset split configuration and dataset shuffle seed. Some of these variables remain the same across all experiments. Some inline comments are included to clarify the purpose of each parameter to assist the user.

```
#command line arguments
parser.add_argument("--model_name", type=str, default="resnet50",
                    choices=["resnet50", "efficientnet_b0"],
                    help="model type to use")#help if user requires
parser.add_argument("--data_root", type=str, required=True,
                    help="path to MURA root")
parser.add_argument("--train_csv", type=str, required=True,
                    help="path to train_labeled_studies.csv")
parser.add_argument("--valid_csv", type=str, required=True,
                    help="path to valid_labeled_studies.csv")
parser.add_argument("--output_dir", type=str, default="checkpoints",
                    help="where 2 save best model")
parser.add_argument("--batch_size", type=int, default=32,
                    help="batch size for training")
parser.add_argument("--num_workers", type=int, default=4,
                    help="number of dataLoader workers")
parser.add_argument("--lr", type=float, default=1e-4,
                    help="learning rate")
parser.add_argument("--epochs", type=int, default=5,
                    help="number of training epochs")
parser.add_argument("--img_size", type=int, default=224,
                    help="input image size")
parser.add_argument("--seed", type=int, default=42,
                    help="random seed")
parser.add_argument("--split", type=str, default="80_10_10",
                    choices = ["80_10_10", "70_15_15"],
                    help="which dataset split to use")
parser.add_argument("--split_seed", type=int, default=67,
                    help="seed for creating file splits")#doesnt need
```

Figure 5.6.1: Command Line Parameters

In addition to this flexible configuration, reproducibility was an important aspect of this project. Deep learning processes contain multiple sources of randomness, these include weight initialisation, dataset shuffling and GPU behaviours. To ensure that experimental results remain consistent across runs, seed control was implemented. This is shown in Figure 5.6.2.

```python
def set_seed(seed: int):#take int from arg parse
    random.seed(seed)#sets seed 4 pyth module
    np.random.seed(seed)#for numPy

    torch.manual_seed(seed)#same seed set for cp
    torch.cuda.manual_seed(seed)# set for gpu
    torch.cuda.manual_seed_all(seed) #multi gpu

    torch.backends.cudnn.deterministic = True#on
    torch.backends.cudnn.benchmark = False#stop
```

Figure 5.6.2: Seed Configuration

As seen in the Figure above, the implementation defines a dedicated seed function that assigns the random generation used by Python, NumPy and PyTorch. The function also sets the seed for CPU operations as well as GPU computations, ensuring that both host and device follow the same random sequence. Additional settings are applied to CUDA to make sure of deterministic behaviour and disable performance optimisations that may introduce non deterministic execution.

These implementations put together ensure that this experimental framework can execute repeatable training runs under precise conditions. This is a vital component of this study because it ensure that all differences are credited to model architecture or dataset split rather than uncontrolled randomness that is present in any machine learning interface.

5.7 Model Checkpoints and Best Epoch Selection

During training, model parameters are updated over multiple epochs as the model continues to learn. Model performance fluctuates during training, this means that the final epoch does not always correspond to best performance. To ensure that the best model configuration is preserved for training, a checkpoint feature was added to monitor validation performance and save the best possible setting.

As seen in Figure 5.7, the training script tracks validation loss after each epoch. The current validation loss is compared with the best validation loss observed thus far. If validation loss for the current epoch is lower than the best validation loss observed, the model is considered to have improved and the checkpoint Is updated.

```python
    if val_loss < best_val_loss:
        best_val_loss = val_loss #check loss this epoch vs best thus far
        best_model_path = os.path.join(args.output_dir, f"{args.model_name}_seed_{args.seed}_best.pth")
        torch.save(model.state_dict(), best_model_path)
        print(f"\nBest model saved at: {best_model_path}\n")#where model is saved on laptp
    #if current training loss is better than best so far, save model weights
print("\nTraining completd")
```

Figure 5.7: Checkpoint for Best Val Loss

When an improvement is observed, the value of 'best_val_loss' is updated and current model parameters are saved to device disk using 'torch.save( ). The checkpoint file name includes the model name and the seed used for that training run. This makes it easy to distinguish between the many experiments.

This design feature helps mitigate the risk of performance degradation potentially caused by overfitting in later training stages while ensuring that evaluation results reflect the highest performing state of the model during training.

The training stop criteria used in this work is minimum validation loss, the checkpoint that corresponds to the lowest validation loss across all epochs is selected as the final model state for evaluation. This was chosen because validation loss provides a direct indication of how well the model generalises unseen data at each epoch, making it a more credible indicator as opposed to training loss alone, which continues to decrease while overfitting occurs (Prechelt, 1998).

5.8 Held-out Test Set Creation

A dedicated test set was created to ensure that the final evaluation of each model was performed on data that was entirely isolated from the training and validation process. The original MURA dataset includes individual training and validation data but does not include an independent test set. To support a complete experimental evaluation, the 2 sets of data provided by MURA were first merged into a large dataset before constructing new splits.
As seen in Figure 5.8.1, training and validation CSV files were loaded and concatenated into a single dataset. Duplicates were removed to ensure each study only appeared once in the large combined dataset. This then allowed new splits to be generated in a controlled manner. Requested split rations were defined through the command line argument '—split', allowing for either an 80/10/10 or a 70/15/15 split.

```
set_seed(args.seed)
if args.split == "80_10_10": #set seed for consistency
    split_ratios = (0.8, 0.1, 0.1)
elif args.split == "70_15_15":
    split_ratios = (0.7, 0.15, 0.15)
else:
    raise ValueError(f"Unknwn split: {args.split}")#check if same as argparse line if not err
print(f"Split for this run {args.split} ratios={split_ratios} split_seed={args.split_seed}")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")#uses gpu to train, or c
print(f"Using device: {device}\n")

train_df0 = pd.read_csv(args.train_csv, header=None, names=["study_path", "label"])#load data
valid_df0 = pd.read_csv(args.valid_csv, header=None, names=["study_path", "label"])

full_df = pd.concat([train_df0, valid_df0], ignore_index=True)#combine mura csv into 1 pool
full_df = full_df.drop_duplicates(subset="study_path").reset_index(drop=True)#remove duplicat
```

Figure 5.8.1: Create Split Ratios and Create Combined Pool

An important aspect of this study was that dataset partitions being created at the patient level rather than at the image level. Figure 5.8.2 illustrates how patient identifiers were extracted and used to allocate entire patients to either training, validation or testing subsets. Splits were precisely created by multiplying the number of each individual cases by the value of each partition, i.e. 0.7 = 70% training. This design prevents data leakage, which could occur if images from the same patient appeared in both training and evaluation subsets. MURA's design increases the likelihood of this occurring, for example patient x may have 3 images in their study, image 1 is seen by the model during training, if image 2 from patient x appears in training the model may be able to classify it accurately during testing simply because it knows from which patient study the image comes from. This would defeat the whole purpose of learning meaningful artefacts from each radiograph. By ensuring that all images belonging to a patient remain within a single subset, evaluation becomes more accurate.

```
n_train = int(split_ratios[0] * n_patients)#takes patients and multipply by ratios
n_val = int(split_ratios[1] * n_patients)#use int to round to whole num
n_test = n_patients - n_train - n_val

train_patients = rng[:n_train]#split by patient id
val_patients = rng[n_train:n_train + n_val]
test_patients = rng[n_train + n_val:]

train_split = full_df[full_df["patient_id"].isin(train_patients)]#extract from big pool
val_split = full_df[full_df["patient_id"].isin(val_patients)]#assign based on split
test_split = full_df[full_df["patient_id"].isin(test_patients)]

print(f"training studies: {len(train_split)}")#print for doubl check
print(f"validation studies: {len(val_split)}")
print(f"test studies: {len(test_split)}")

train_out = train_split[["study_path", "label"]]#narrow don to study path and label
val_out = val_split[["study_path", "label"]]
test_out = test_split[["study_path", "label"]]

train_csv_path = os.path.join(args.output_dir, f"train_{args.split}_splitseed{args.split_seed}.csv")#
val_csv_path   = os.path.join(args.output_dir, f"val_{args.split}_splitseed{args.split_seed}.csv")
test_csv_path  = os.path.join(args.output_dir, f"test_{args.split}_splitseed{args.split_seed}.csv")
```

Figure 5.8.2: Creation of Test Set

To maintain reproducibility, dataset partitioning process was controlled by '—split_seed'. This seed was set to 67 and did not change throughout the whole experimental process. This seed

determines the order of patients before the split is applied, allowing identical train, validation and test sets can be recreated across all runs. The design allows results to be interpreted as a result of architectural differences rather than variations in splits.

Once partitions were made, each split was exported to individual CSV files and saved on disk. This step freezes the held out test set, make sure that it is untouched throughout model development and other experiments. By doing this the final evaluations in Section 7 reflect a genuine measure of a model's ability.

Overall, this implementation provides a controlled and reproducible framework. By merging original MURA files, applying patient level splitting, controlling dataset shuffling and exporting the frozen test set. The pipeline makes sure that final model conclusions are entirely independent from the training process.

5.9 Grad-CAM Implementation

To add qualitative interpretation of the CNNs in this dissertation, Grad-CAM was implemented. This provides a visual explanation of a models predictions by highlights the specific regions of an image that are the strongest contributors to a model's decision making. Specifically, in this study Grad-CAM was used to analyse how models focus on anatomical structures when predicting whether an image belongs to the abnormal or normal class.

This begins by attaching hooks to a chosen target layer in the model, allowing activations and gradients to be captured during interference. As seen in Figure 5.9.1, a forward hook records the feature maps produced by the target layer during each forward pass. A backward hook captures the gradients flowing through the same target layer, but during backpropagation.

```
def register_hooks(self):
    def forward_hook(module, input, output):#triggers on forward pass
        self.activations = output.detach()#saves ouput to self activ

    def backward_hook(module, grad_input, grad_output):#trig during back pass
        self.gradients = grad_output[0].detach() #get gradie score for layer

    self.handles.append(#handle on and off for layer
        self.target_layer.register_forward_hook(forward_hook)
    )
    self.handles.append(#handle on/off for back layer
        self.target_layer.register_full_backward_hook(backward_hook)
        #keep check for update pytorch models
    )

def remove_hooks(self):
    for handle in self.handles:#iterate thru handles and remove
        handle.remove()# clear handle
    self.handles = []

def generate(self, input_tensor, target_class = None):
    self.model.eval()#sctivate eval mode
    self.model.zero_grad()#clear old grad to avoid accum

    output = self.model(input_tensor)#runs input through model to get pred

    if target_class is None:#if not specifiied, pick highest prediction
        target_class = output.argmax(dim=1).item()

    score = output[:, target_class]#isolate pred for targ layer
    score.backward()#triggers back prop, causes back hook to capture gradien
```

Figure 5.9.1: Grad Cam Target Layer Selection and Hook Creation

These 2 signals are essential for computing Grad-CAMs because the represent both the spatial features extracted by the model as well as the importance of the extracted features for the predicted class.

The model first performs a standard forward pass to obtain the output logits for the image. If no target class is mentioned, the class with the highest predicted probability is chosen. Backpropagation is performed in accordance to the score of this target class, causing gradients to flow back through the network and trigger the already registered backward hook. It is important to note that the sensitivity of the predicted class score exist with respect to the feature maps of the target layer.

Grad-CAM is computed by performing global average pooling over the spatial dimensions of the gradients, creating a set of channel oriented weights. Each weight lies in accordance of importance to its corresponding feature map for the predicted class. These weights are multiplied with the stored activations and calculated across channels to produces a localisation map. A ReLU activation is applied; seen in Figure 5.9.2, to keep positive contributions only, because of this the heatmap only highlights regions that positively influence model decisions.

```
weights = self.gradients.mean(dim=[2, 3], keepdim=True)#whi

cam = (weights * self.activations).sum(dim=1, keepdim=True)
cam = F.relu(cam)#to keep +ve influence

cam = F.interpolate(#stretch layer to size of img
    cam,
    size = input_tensor.shape[2:],
    mode = 'bilinear',#using this interpolation
    align_corners = False
)

cam = cam.squeeze().detach().cpu()#remove xtra dim
cam -= cam.min()#shift values to lowest 0
cam /= (cam.max() + 1e-8)#to prevent crash if div 0

return cam, output
```

Figure 5.9.2: ReLU function for GradCAM

Feature maps from deep layers have lower spatial resolution than the original image, the resulting activation map contains more samples more samples, bilinear interpolation is used to match image size. An important normalisation step is carried out to allow proper visualisation across different images and predictions.

All these factors put together allow assessment of whether models rely on useful features or focus on irrelevant structures instead. The actual visualisations are seen in Section 6.8

6 Testing and Evaluation

6.1 Evaluation Setup

The evaluation stage of this study follows the experimental design established in the Methodology chapter. Both models were evaluated under identical conditions using MURA across 2 dataset splits, 3 fixed random seeds and patient level partitioning. To account for the stochastic nature of neural network training (LeCun, et al., 2015), experiments were repeated across multiple seeds to ensure conclusions are not taken from a single randomly initialised run. Model performance was assessed using the checkpoint with the lowest validation loss with a heavy emphasis on abnormall class recall at study level as the primary metric because of the clinical consequences of false negatives (Hicks, et al., 2022).

6.1.1 Preliminary Pilot Runs

Before the final controlled experiments were conducted, 2 pilot runs were carried out during the early stages of development. The purpose of these runs was to make sure that the data loading pipeline, training procedure and evaluation code was functioning as intended before deploying final test runs including the full experimental configuration. During the pilot runs the original MURA dataset organisation was applied; meaning uncontrolled randomness in weight initialisation and dataset shuffling. Additionally, a separate test set had not been created. The focus of these pilot runs (run_id 1 and 2) was to confirm that the training and validation pipelines operated as expected, these runs did in no way contribute to model analysis.
During these initial runs, an issue was identified in the study level evaluation metrics. See Figure 6.1.

```
Study-level classification report:
{'0': {'precision': 0.0, 'recall': 0.0, 'f1-score': 0.0, 'support': 1.0}, '1': {'precision': 0.0, 'recall': 0.0, 'f1-score': 0.0, 'support': 0.0}, 'accuracy': 0.
0, 'macro avg': {'precision': 0.0, 'recall': 0.0, 'f1-score': 0.0, 'support': 1.0}, 'weighted avg': {'precision': 0.0, 'recall': 0.0, 'f1-score': 0.0, 'support':
 1.0}}

Best model saved at: ..\checkpoints\resnet50_best.pth
```

Figure 6.1: Corrupted/Missing Study Level Metrics

Initially, it appeared that the absence of the study level metrics was simply an effect of early training behaviour. However, this pattern carried over even into later epochs; further investigations revealed that the metrics were incomplete because images belonging to the same study were not being grouped correctly during evaluation. This occurred because the dataset class did not consistently return a stable study identifier for each image sample. This meant that predictions from images belonging to the same study could not be aggregated accurately.
After identifying the issue, the dataset class was modified to make sure that each images returned a consistent study_id. Once this change was made, image predictions were grouped correctly and study level aggregation operated as originally expected.

Since pilot runs were carried out before final experiments, they are not part of the main comparative analysis presented in this dissertation. Instead, they served as an important validation step that helped refine the experimental pipeline prior to carrying out final sets of controlled experiments.

6.2 Baseline Training Behaviour

To understand how architectures learn during training, the evolution of training and validation losses was analysed across the 10 training epochs. Figure 6.2 illustrates the mean training and validation loss for ResNet50 and EfficientNet-B0 averaged across the 3 chosen seeds. Observing this allows us to understanding training stability, convergence behaviours and identify potential overfitting before comparing final performance metrics. Commencing from this Figure onwards, all computations are carried out only on the actual controlled experiments (run 3 to 14)

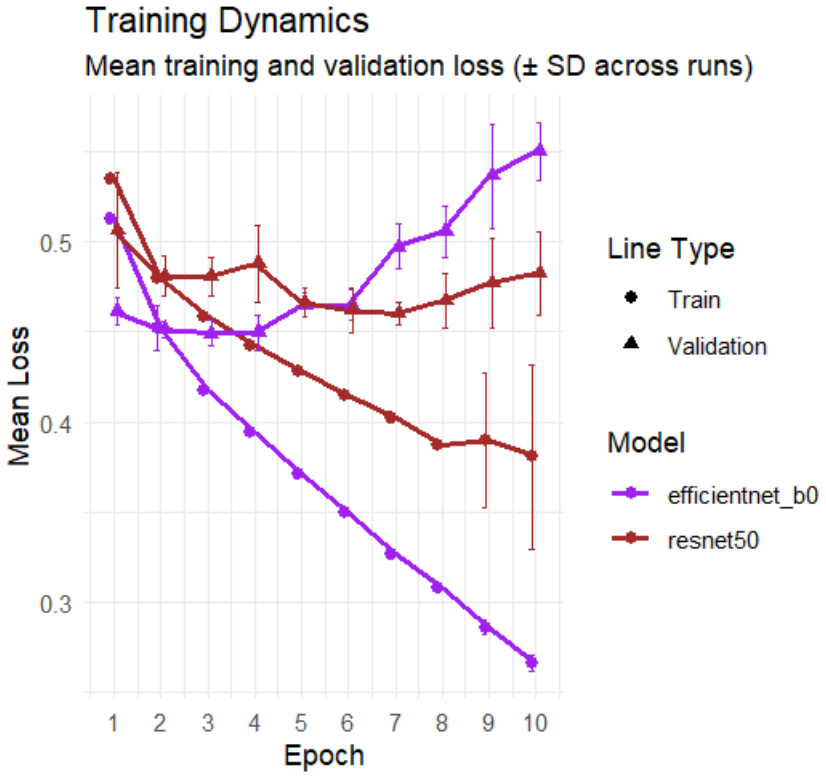


Figure 6.2: Mean Training and Validation Loss Curves across All Seeds for ResNet50 and EfficientNet-B0 During the 10 Epoch Training Process (error bars show ±1 SD across runs)

The Figure shows that initially, both architectures start with similar loss numbers during the first epoch. This shows us that both models start training from somewhat similar optimisation states after transfer learning initialisation. As training progresses, training loss steadily decreases for both models. Showing that both architectures are capable of learning meaningful content from the images.

It is clear that the convergence rate differs between the 2 architectures, EfficientNet-B0 shows a much clearer decline in training loss, reaching far lower values than ResNet50 by the final few epochs. This may suggest that EfficientNet-B0 is more efficient in adaption to the MURA dataset during fine tuning. On the other hand, ResNet50 demonstrates a slower and more gentle decline in training loss, indicative of a more conservative optimisation course.

The validation curves reveal a different pattern. Despite training loss consistently decreasing for both architectures across epochs, validation loss almost seems to stabilise after early epochs and begins to diverge from the training lines. This may suggest that additional epochs mainly improve the models' ability to fit training data rather than actually improving generalisation to unseen data. EfficientNet-B0 shows a more visible increase in validation loss during later epochs, showing that even though this model learns fast, it also begins to overfit data more than ResNet50.

The error bars provide more insight into the consistency of performance across different runs. The relatively small errors bars seen in early epochs show that both models exhibit mostly consistent behaviour to begin with. However, as epochs pass, variability also increases, particularly loss. This is more clear in EfficientNet-B0's case where later epochs display larger error bars suggesting a greater sensitivity to factors like randomness and dataset splits. Alternatively, ResNet50 shows more stability in loss values, shown by narrower error bars across a majority of epochs.

The divergence seen between training and validation loss seen in this Figure implies a lot about training duration. For EfficientNet-B0, validation loss begins to rise noticeably from around epoch 6 onwards despite the continued reduction in training loss, telling us that further epochs primarily overfit training data rather than improving generalisation.

ResNet50 shows a similar pattern, albeit more gradual. This may suggest that both architectures might benefit from an early stopping function based on validation loss and this would have also assisted in identifying an optimal stopping point earlier than the 10 fixed epoch limit in this dissertation. The checkpoint mechanism for best model weights partially addresses this problem by allowing for final evaluation to use the best model state. An early stopping mechanism would have been more beneficial and is consistent with best practice in deep learning (Prechelt, 1998).

The checkpoint mechanism described earlier ensure that the model state used for evaluation is not the final epoch, but the epoch with the lowest validation loss. For EfficientNet-B0 this corresponds to epochs 3 or 4 across all runs and for ResNet50 epochs 6 through 8, both of which fall before significant validation loss is visible in the Figure above. These results allow analysis in further sections reflect a model's best state rather than an overfitted config.

Overall, both models show stable convergences through the training process, confirming to us that the optimisation procedures are functioning correctly. These observations will provide useful context for subsections to follow, where final evaluation metrics and architectural performance differences are analysed further.

6.3 Architecture Performance Comparison

The main objective of this dissertation is to compare how ResNet50 and EfficientNet-B0 react when identifying abnormal musculoskeletal studies under controlled training conditions. Earlier, it was established that the main evaluation metric in this dissertation is recall for abnormal cases at the study level, given that failure to detect an abnormal case carries a larger consequence. Figure 6.3 compares mean abnormal recall achieved by both architectures, averaged across all seeds and dataset split configs.

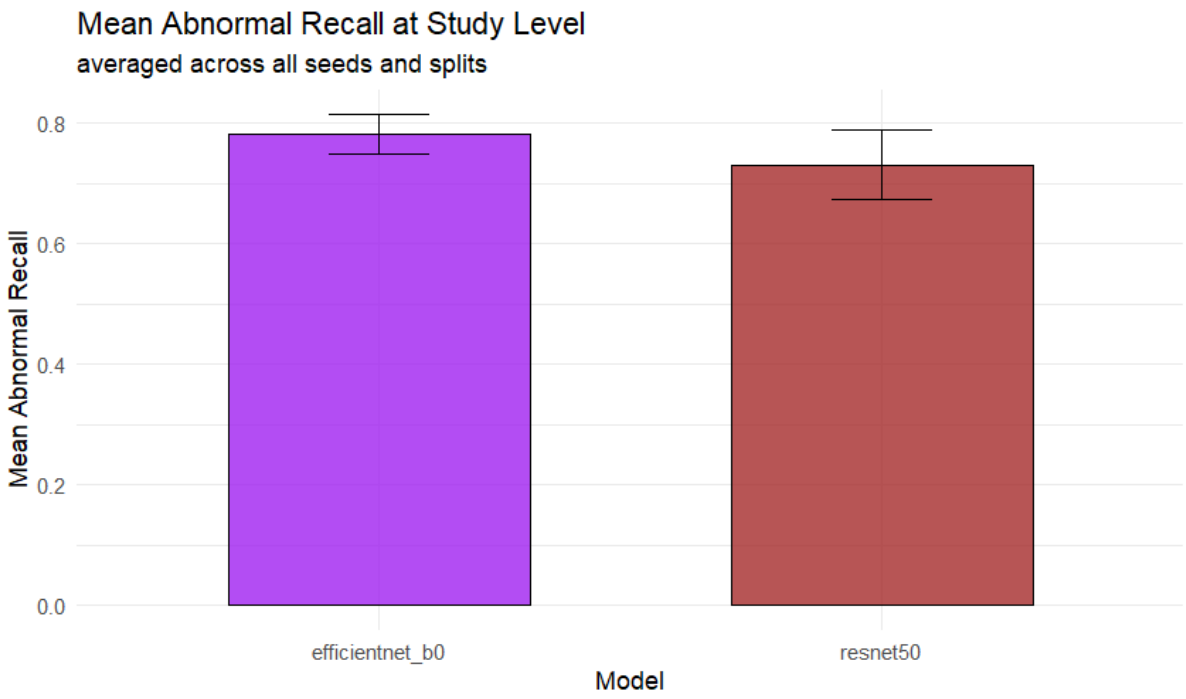


Figure 6.3: Mean Study Level Abnormal Recall for ResNet50 & EfficientNet-B0 across all Seeds and Dataset Splits – validation set (error bars represent ±1 SD)

The Figure shows that EfficientNet-B0 achieves the overall highest mean study level abnormal recall (average value of approx. 0.78) compared with a lower mean of approximately 0.73 for

ResNet50. Under the given experiment conditions, this shows that EffiicientNet-B0 is slightly more effective at identifying abnormal cases. In simple terms, this means that EfficientNet-B0 misses out on fewer abnormal studies on average, making it the better architecture with regards to the primary metric of this study.

The Figure above also shows a clear difference in model performance stability. ResNet50 has wider error bars than EfficientNet-B0, displaying a greater variation across seeds and splits. This tells us that ResNet50 performs less consistent under stochastic training variations, EfficientNet-B0 shows more stable behaviour across runs. It is important to understand the scale of the difference, the mean gap in study level abnormal recall between both models is approximately 0.01 across all runs, this is a very small margin. The importance of this finding is not in the size of gap but in how frequently it occurs. EfficientNet-B0 performs better than ResNet50 on study level abnormal recall in every single run and configuration of dataset split and seed. The probability of this occurring coincidentally under the assumption of small architectural differences is low. This tells us that the gap between the abnormal recall at the study level reflects a genuine property of EfficientNet-B0's design rather than a simply a statistic. This distinction is of significance because in a medical context, a higher likelihood to detect an abnormal case, even if it is by a very small margin carries practical significance, which is not captured by simply aggregating mean values.

This is an important observation given that the use of multiple seeds was intended to avoid conclusions being drawn from a single run. The lower spread observed for EfficientNet-B0 illustrates that not only did the model perform better than ResNet50 on abnormal recall, but its performance was also more consistent and stable, making that advantage more reputable.
In an architectural analysis context, this behaviour is likely. EfficientNet models as a family are designed around compound scaling and parameter efficient feature extraction. Giving EfficientNet-B0 the ability to learn useful representations without relying solely on increased model depth. This study shows that this design complex appears to support more coherent abnormal case sensitivity. ResNet50 does not fall far behind and still achieves strong values, but given its lower mean recall and larger room for error, showing that it is less effective at maximising study level abnormality accuracy.

This result is important challenges the effectiveness of this experimental framework. Since abnormal recall scores reflect a model's ability to accurately detect abnormal studies, the higher recall scores achieved by EfficientNet-B0 show that it offers more reliable behaviours, given the conditions of this dissertation. However, this figure alone cannot be interpreted as a deciding

factor in which architecture is superior. It does not illustrate whether this accuracy comes at the cost of other metrics such as precision. It also does not illustrate whether accuracy is equally strong across both dataset splits. These questions are addressed in the following sections through split specific analysis, seed stability analysis and further analysis of abnormal class detections.

It is also worth noting that the compound scaling mechanism of EfficientNet-B0 is a potential strong contributor to good recall scores, it is not the sole factor. The SE attention mechanism allows channel wise feature recalibration, this can allow the model to select task relevant frequency bands within the monochrome look of radiographic images. ResNet50 does not possess this same advantage. The Grad-CAM interpretations in Section 6.8 provide fair qualitative evidence of this.

6.4 Effect of Dataset Partitioning

The influence of dataset partitioning on abnormal detection performance was examined by comparing the results of the 2 dataset split configurations. 80/10/10 and 70/15/15 for training, validation and testing respectively. While both of these maintain patient level separations between subsets, they differ into the amount of data allocated to each subset. The 80/10/10 split provides a larger training set, while the 70/15/15 split increases the amount of data in the validation and testing subset. Figure 6.4 illustrates the mean abnormal recall achieved at the study level by each model under both dataset splits, averaged across the 3 seeds.

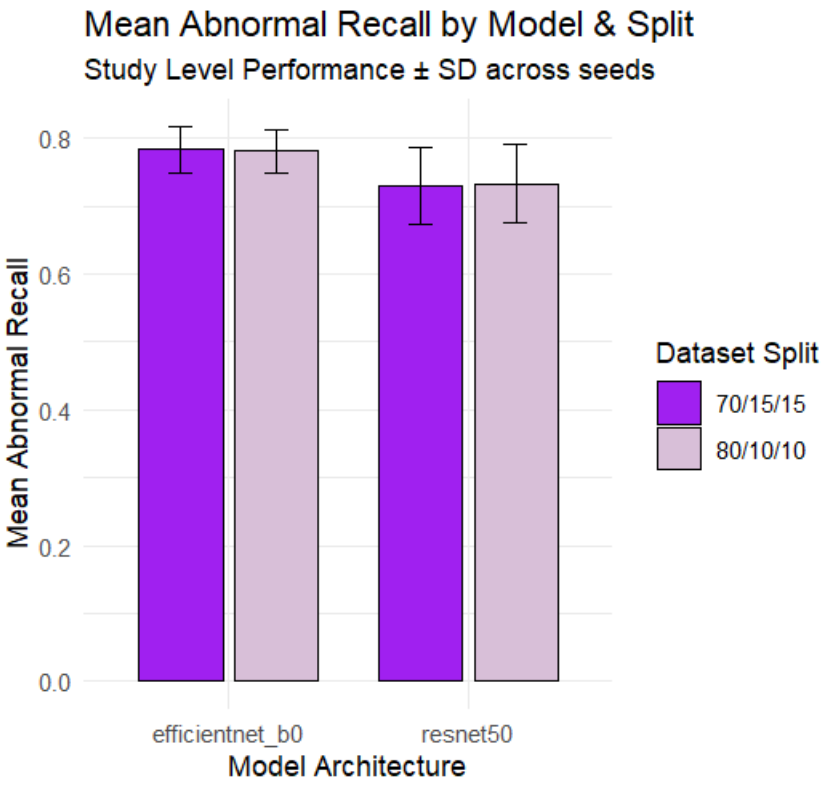


Figure 6.4: Mean Study Level Abnormal Recall by each Architecture under the 2 Dataset Splits (error bars represent ±1 SD across the 3 seeds)

As show in Figure 6.4 above, the effect of dataset splitting on abnormal recall is minimal for both models. EfficientNet-B0 displays almost identical recall values under both split configurations, with mean recall hovering around 0.78 irrespective of whether 70 or 80% of the data is allocated to training. ResNet50 also shows virtually no change in abnormal recall between the 2 splits, sitting at an average approximate of 0.73 across configurations. The error bars that represent standard deviation across seeds seem to overlap for both models, indicating that differences are small in relation to the natural variability induced by stochastic training methods.

Table 6.1 provides the exact abnormal recall values that make up the results shows in Figure 6.4.

	model	split	mean_recall	sd_recall
1	efficientnet_b0	70/15/15	0.7831623	0.03349108
2	efficientnet_b0	80/10/10	0.7814414	0.03160786
3	resnet50	70/15/15	0.7296427	0.05715320
4	resnet50	80/10/10	0.7327862	0.05825335

Table 6.1: Mean Study Level Abnormal Recall Across Dataset Splits (rounded to 4dp)

Albeit the bar chart suggest near identical performance, the table confirms that small numerical differences exist. The differences are very minimal , with values varying by less than 0.004 for

both architectures across both splits. Alternatively, standard deviation across seeds is larger, ranging between approximately 0.03 to 0.06. This shows that stochastic training variation contributes more towards performance variety rather than the dataset split strategy itself. As a result, the abnormal detection values of both architectures can be considered mostly stable, accounting for minimal changes in dataset split settings.

A closer look of the split results shows a small but interesting observation. Both architectures do not have a noticeable difference in reaction to the choice of split. EfficientNet-B0 demonstrates a minimal improvement under the 70/15/15 configuration compared to ResNet50. Albeit minimal, the pattern exists, this can suggest that EfficientNet-B0's parameter efficient design is less reliant on the amount of training data to complete the task at hand. ResNet50 has more parameters and depth in its design, it may require significantly more training data to fully make use of its generalisation capabilities. Literature also shows that networks with a deeper design show stronger reliance on data that can be fine-tuned (Yang, et al., 2022). This outlook can be considered speculative at best given the relatively small number of combinations tested in this dissertation, the overlap in standard deviations across seeds means absolute conclusions cannot be made. However, it opens up a path to further investigation and adds some variability to a mostly straightforward statement that split strategy has limited influence on the task.

The results in Figure 6.4 suggest that the ability to capture abnormal cases by both architectures is largely robust when reacting to moderate changes in dataset splits. Despite increasing the amount of data available to the training subset, this increase does not appear to generate any meaningful improvement in abnormal recall with relation to the MURA dataset and training procedures employed by this dissertation. It can also be concluded that increasing the size of the evaluation subset does not alter abnormal recall in a substantial manner.
From a design perspective, this observation strengthens the reliability of the model comparison presented in Section 6.3. Given that both dataset splits produce near identical abnormal recall values, the visible performance advantage that EfficientNet-B0 displays cannot be concluded as a result of a particular dataset split. However, it may be representative of a genuine differences in how the 2 models respond to the task under the given experimental conditions.
In conclusion, analysis tells us that dataset split choices have limited influence on abnormal recall at the study level within the tested experiments, indicating that comparative conclusions drawn in this study remain stable throughout both dataset splits.

6.5 Seed Stability Analysis

Neural network training contains inherent stochastic components that originate from factors like weight initialisation, dataset shuffling, order of mini batches and optimisation dynamics. If not particularly set, the same model trained under identical conditions can produce slightly different readings across different runs. To account for these variations and ensure that observed architectural differences are not a result of a favourable random initialisation, each experiment in this dissertation was repeated using 3 seeds. Analysing performance variation across these 3 seeds allows the stability of each architecture to be assessed as seen in Figure 6.5:

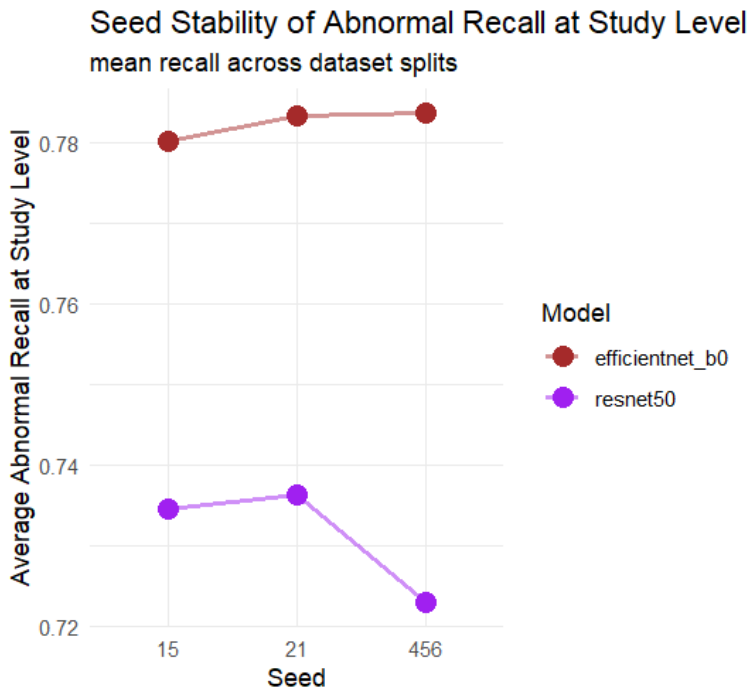


Figure 6.5: Abnormal Recall achieved at Study Level Across the 3 seeds for each Architecture (points represent mean recall across both dataset splits)

The Figure above shows the mean study level abnormal recall recorded for each seed after averaging across both dataset splits. A number of patterns can be seen, firstly EfficientNet-B0 demonstrates highly consistent performance across all seeds. The abnormal recall values remain tightly clustered around the range of approximately 0.78 and 0.785, this tells us that the architecture produces similar outcomes regardless of the random initialisation present during training. The limited spread suggests that EfficientNet-B0 is quite robust to stochastic training effects within the experimental conditions used in this project.

Alternatively, ResNet50 exhibits greater variation across seeds. The results for seed 15 and 21 remain close to each other within the range of 0.734 to 0.737, seed 456 drops to approximately 0.723. Even though this difference is not large in the grand scheme of things, it represents a

large fluctuation in relation to the behaviour of EfficientNet-B0. This pattern shows us that ResNet50 is likely more sensitive to stochastic training conditions, meaning that different random conditions may lead to varying abnormality detection outcomes.
Another difference is that EfficientNet-B0 consistently achieves higher abnormal recall than ResNet50 across all seeds. ResNet50 does not surpass EfficientNet-B0 a single time even stochastic variation is considered. This is important because it suggests that EfficientNet-B0's advantages, that were identified earlier in this study, are not dependent on a particular seed. Indicating that these results appear to reflect an actual systematic difference in how the 2 models respond to this image classification task.

The standard deviation in study level abnormal recall across all runs for EfficientNet-B0 is 0.0182 compared to 0.0252 for ResNet50, this is about a 28% decrease in variance between models. This difference reinforces the observation that EfficientNet-B0 produces more predictable outcomes under stochastic initialisation. It is hard to pinpoint why ResNet50 is more sensitive to seed 456, random seeds influence the way in which model weights are initialised and mini batch ordering. It is possible that ResNet50's deeper architectural design creates a more complicated optimisation base. For example, certain initialisations lead the model to output poor trajectories. EfficientNet-B0 has a more restricted search space meaning it is less susceptible to the same. Literature supports this claim by suggesting that deeper residual networks show higher sensitivity to initialisation as opposed to architectures that make use of compound scaling (Tan & Le, 2019). However, this claim is also speculative without further experimentation such as ablation or explainable AI (XAI) experiments.

Overall, this seed stability analysis reinforces the reliability of earlier findings, EfficientNet-B0 not only achieves higher mean abnormal recall but accompanies it with higher consistency across stochastic training conditions. ResNet50 demonstrates more variability between runs. These observations back up the conclusion that EfficientNet-B0 provides a more stable basis to detect abnormal recall at the study level.

6.6 Abnormal Case Detection Performance

Abnormal recall is an important measure of a model's ability to identify an abnormal study, however it does not fully describe abnormal detection patterns on its own. A model can achieve higher recall by simply predicting a larger number of cases as abnormal, hence increasing the number of False Positives. For this reason, abnormal precision must be also considered with abnormal recall to understand whether improvements in detection come at the cost of a lower classification specificity. Figure 6.6 compares abnormal class precision and abnormal class

recall, both at the study level achieved by each model averaged across all seeds and dataset splits.

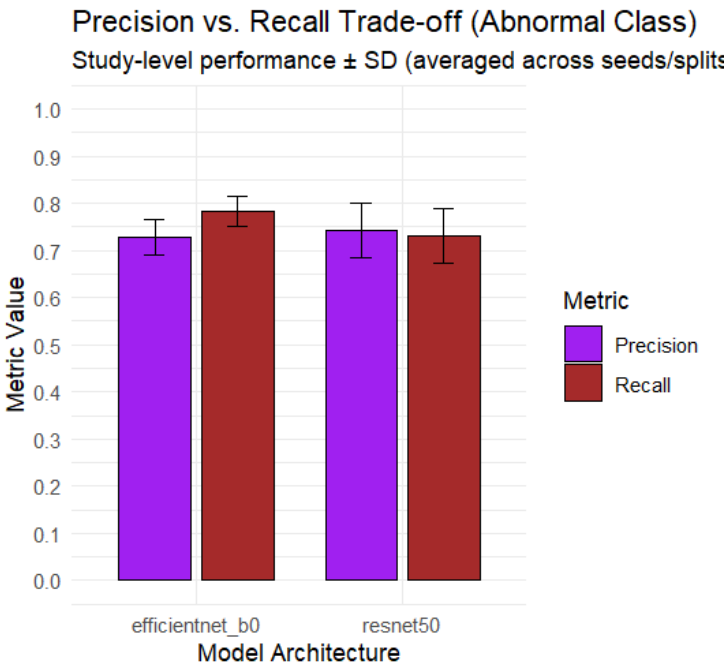


Figure 6.6: Comparison of Abnormal Class Precision and Recall at Study Level (by each model, averaged across all seeds & dataset splits)

As shown above, EfficientNet-B0 outputs a slightly higher abnormal recall with values of about 0.78 compared to slightly lower values achieved by ResNet50 that hovers in the 0.72 to 0.74 range for ResNet50. This finding lines up with earlier architectural comparison that confirms EfficientNet-B0 detects a larger portion of abnormal studies as a whole. In this medical context, this tells us that EfficientNet-B0 outputs fewer False Negatives, meaning that abnormal cases are less likely to be missed.

It is also important to understand the increase in recall does not appear to come at the opportunity cost of precision. The abnormal precision values for both architectures are mostly similar, EfficientNet-B0 achieves values in the range of 0.71 to 0.73, while ResNet50 shows slightly higher values in the range of 0.74 to 0.75. The error bars in the Figure above that represent standard deviation across seeds demonstrate an overlap, implying that the difference in precision for the abnormal class is relatively small between the 2 architectures, with respect to the variability present due to stochastic training effects.

This behaviour highlights the typical trade off present between precision and recall in most classification systems (Buckland & Gey, 1994). EfficientNet-B0 appears to favour higher abnormal recall sensitivity, meaning that it identifies a larger number of abnormal cases but may occasionally flag additional normal studies as abnormal. ResNet50 demonstrates higher

abnormal class precision, telling us that its abnormal class predictions are slightly more conservative. It is important to note that the increase in precision is accompanied by lower recall, meaning that a larger number of abnormal cases are missed on average.

In a medical context this trade-off is of significance, since failing to detect an abnormal case bears greater consequence, the higher recall achieved by EfficientNet-B0 illustrates a more favourable pattern within this study's experimental setting.

It is interesting that the precision to recall relationship is not entirely similar across all experimental configurations. ResNet50 achieves higher study level abnormal recall than EfficientNet-B0 under the 80/10/10 split. This means that the overall advantage that EfficientNet-B0 seems to possess thus far is partially due to its scores using the 70/15/15 split. This behaviour adds an important angle to the larger abnormal precision and recall comparison, EfficientNet-B0's advantage in abnormal sensitivity is more noticeable when the evaluation dataset is larger. This may point to its predictions being more stable and less influenced by smaller test subsets. The precision difference is approximately 1.7% between the 2 models on the separate test set. A simpler way of illustrating this is that for every 100 studies classified as abnormal, EfficientNet-B0 outputs about 2 more false positive predictions than ResNet50. This small increase in false positives represents a potentially reasonable trade off against the simultaneous increase in true abnormal detection, this judgement would depend entirely on the context of the specific clinical setting.

As a whole, the results suggest that EfficientNet-B0 provides a more favourable balance between abnormal class recall and precision, reinforcing the architectural EfficientNet-B0 seems to possess thus far.

The relatively high abnormal recall observed across both models encourages a deeper examination of this metric. Several factors contribute to recall sitting at this level. The max aggregation rule is a key player because it classifies an entire study as abnormal if any single image receives an abnormal prediction, the aggregation strategy is heavily biased toward higher recall by design. This means that a fair proportion of the abnormal recall improvement seen at the study level is a product of the evaluation choices rather than architectural design, discussed further in Section 6.7. A second contributing factor is the usage of ImageNet pretrained weights which gives the architectures strong low level feature representations that support sensitivity to structural abnormalities from the start of training. Although it is important to note that ImageNet does not provide direct support for X-rays since it is a dataset that comprises of

everyday objects (Deng, et al., 2009) rather than medical data. Regardless, transfer learning has been shown to improve abnormal class sensitivity in medical imaging tasks by reducing the number of individual examples in training need to learn relevant features (Raghu, et al., 2019). The final likely factor is data augmentation during training, random horizontal flips and colour jitter introduce controlled variance that encourages both models to learn more generalisable abnormal patterns rather than overfitting to specific image traits.

Despite these factors, improvements are achievable. The most targeted means to do this would be by adding a class weighted loss function during training which assigns a higher penalty to false negative predictions as opposed to false positives. This supports the clinical priority of reducing missed abnormal cases rather than relying on methods such as interpretability to recover misclassifications. This is seen in (He, et al., 2021), the use of a weighted cross entropy function demonstrated improvements in abnormal class recall, suggesting that this would be an evidence based improvement to the current framework. Another possible option would be to replace the max aggregation rule with a learnable aggregation function which would allow the aggregation strategy to be optimised for recall rather than relying on a fixed and strict rule. For example, a study level classifier trained on image level probability distributions.

6.7 Study Level vs Image Level Performance

In the MURA dataset, diagnostic labels are assigned at the study level, each study may contain multiple images of the same body part. CNNs operate on individual images during training and predictions. To align a model's outputs with MURA's labelling structure, predictions should be aggregated across images belonging to the same study. In this dissertation, study level predictions are generated by applying an aggregation function that combines singular image predictions to output a final study classification. Analysing the difference between image level and study level performance; as seen in Figure 6.7 gives us insight into how application of this aggregation function influences abnormal class detection behaviour.

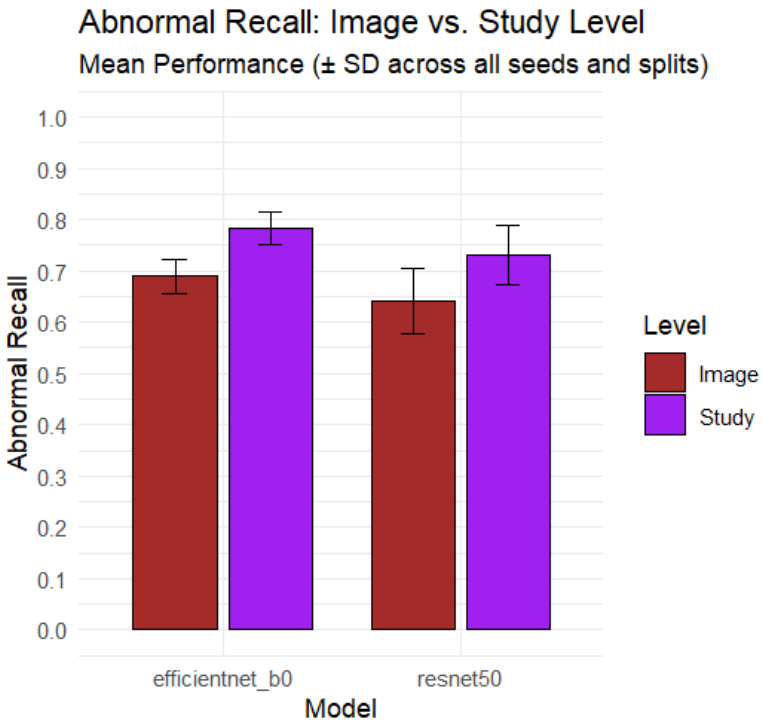


Figure 6.7: Image Level vs Study Level Abnormal Recall for Both Models

The Figure above compares abnormal recall measured at the image level with the recall obtained after applying the study level aggregation function, in both cases study level evaluation produces higher abnormal recall than at the image level. EfficientNet-B0 improves from around 0.69 abnormal class recall at image level to about 0.78 at the study level, ResNet50 also displays a similar trend increasing from about 0.65 to 0.73. These improvement show that aggregating predictions across many images allows the model to recover abnormal cases that were potentially missed when individual images were being evaluated individually.

This behaviour can be backed up by the structure of radiographic imaging. A single abnormality may only be visible in one of the images within a given study. When predictions are evaluated at the image level, each individual image must be correctly identified by the model to achieved a high recall value. Study level aggregation allows the presence of an abnormal prediction in any image within the study to contribute to the final recall score. This means that if the model fails to detect abnormality in some images, it can still accurately class the study as a whole.

The performance difference between the 2 models remains the same after aggregation. EfficientNet-B0 still achieves a higher abnormal class recall than ResNet50 at image and study level, this tells us that EfficientNet-B0's observed advantages in previous Sections is not the result of the aggregation method used. Instead, this aggregation function benefits both models equally by improving abnormal detection.

It is important to note the scale of this improvement, study level aggregation increases abnormal recall by around 8 to 9 percentage points for both architectures, as compared to image level evaluation. This increase also demonstrates that evaluating individual images alone is not a true reflection of abnormality detection capabilities of a model, especially when applied to radiography datasets with multiple images per study. The results provide proven validation for the aggregation strategy explained in Section 4.3. It also confirms that this aggregation step not only improves evaluation quality but also improves the reliability of abnormal detection at the study level; maintaining clinical relevance.

The asymmetry between how much each architecture benefits from study level aggregation is worth noting. EfficientNet-B0 gains 0.0866 from aggregation while ResNet50 gains more at 0.0948 on the test set. This suggests that ResNet50 makes more recoveries of missed abnormal cases through the aggregation step despite its weaker predictions at the image level. This might be indicative of the way that ResNet50 distributes image level error, for example, when the model misclassifies an abnormal image, the other images within the study are more likely to be correctly classified by the max aggregation rule. In contrast, EfficientNet-B0 may produce more consistent but occasionally wrong predictions across all images in a study, leaving less room for recovery by aggregation. This difference shows how different architectural designs interact with the aggregation strategy, it also suggests that image level performance by itself is not an appropriate criterion by which to select a model for projects where multiple images exist in a study.

This observation allows for a deeper examination of the max aggregation function. By classifying an entire study as abnormal if any image in the study is classed as abnormal. This approach is rather biased towards higher recall at the expense of precision. This design choice was clinically motivated, but it means that the study level recall improvements seen in this dissertation are mostly a result of the aggregation functions nature rather than architectural capability. A mean probability function or majority voting function would likely produce different precision to recall values for both architectures and could have also altered the observed advantages. This represents a limitation of the experimental design the future work should take into account.

In conclusion, this Section demonstrates that study level aggregation improves abnormal recall for both architectures. These results support the design choice to prioritise analysis at the study level when assessing abnormal detection performance in this study.

6.8 Model Interpretability (Grad- CAM)

While the qualitative analysis discussed in the above sections is imperative to this study, it does not reveal exactly how models come to their conclusive predictions. CNNs are often criticised for operating as black box systems (Azam, et al., 2023), their internal decision making process is often difficult to interpret. In the medical imaging world, understanding which image regions dictate a model's prediction is imperative, this is because interpretability helps us understand whether a model is relying on clinically meaningful features or spurious correlations as discussed in the Background Section and in (Geirhos, et al., 2020).
This study attempts to provide insight into the decision making behaviour of the trained models. This is done via Gradient weighted Activation Mapping (Grad-CAM).
Grad-CAM has also shown effectiveness across classification tasks in fine grained visual recognition (Selvaraju, et al., 2017), a property that is required when deriving subtle features present in radiographic detection.

This visualisation method generates visual explanations by highlights regions of a given image that have the strongest influence over a model's decision. This is achieved by computing the gradient of the predicted class score with respect to the feature maps of the final layer in each model. These gradients are then used to weight the corresponding feature maps, creating a heatmap that tells us which specific regions contribute the most.

By overlapping the resulting heatmap onto the original image, Grad CAM allows the areas of the highest model attention to be visualised. In this context, these visualisations can help understand whether a model is focused on the anatomically relevant bone structures when identifying an abnormality.

Visualisations were generated using model weights from a single training run, selected based off the already saved best performing checkpoints; decided based on lowest validation loss. A fixed random seed was used for each model to allow for accurate reproducibility of heatmaps. Albeit, multiple training runs were used for quantitative analysis, a single run was used for interpretability analysis to provide a consistent and solid basis for comparison between models. The examples in this Section will display cases of correct abnormal predictions, correct normal predictions as well as an incorrectly classified case for both models. This will allows qualitative insight into how each model uses image features during classification and assist in identifying limitations in model reasoning

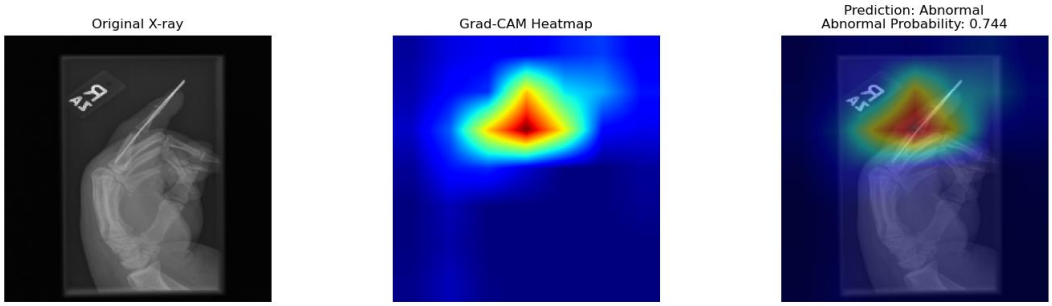


Figure 6.8: Correct Abnormal Classification (EfficientNet-B0)

Figure 6.8 presents an example of a correctly classified abnormal study produced by EfficientNet-B0. The probability score of 0.744 indicates the model's fair confidence in its prediction.

The highlighted regions are concentrated in the upper middle section of the image, lying in accordance with the area containing the finger structure. It is important to note the splint in the image was likely used to stabilise the finger during imaging. The concentrated areas suggest that the model is responding to the correct structural or contextual cues associated with abnormality.

The spread of the heatmap indicates that the prediction is driven by specific features rather than more generic and contextually irrelevant patterns. The highest activation occurs around the splint suggesting that the model may extract features from this area when forming its prediction.

The visualisation also reveals a potential limitation. The strongest activation appears to correspond with the splint more than the abnormality within the bone structure. This tells us that the model could be reliant on contextual indicators like medical supports that are most likely to be found in abnormal cases in the dataset. In this case, this does not affect the accuracy of the prediction in this case, such reliance could reduce robustness if similar contextual identifiers are absent in other images.

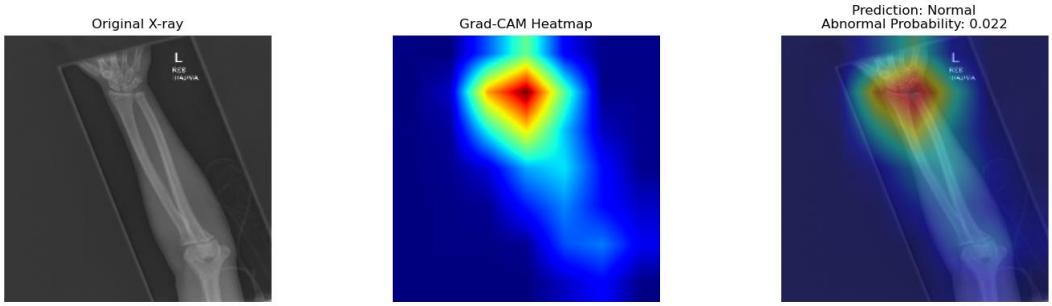


Figure 6.9: Correct Normal Classification (EfficientNet-B0)

Figure 6.9 shows a correctly classified normal case study. The model outputs a low abnormal probability of 0.022 indicating that is it highly confident this image is a normal case.
Even though, the image originates from the 'XR_FOREARM' file in the dataset, the heatmap's activation appears to focus on the wrist region where multiple bones intersect and structural irregularities would most likely be present (McCarthy & Frassica, 2014). This suggests that the model prioritises anatomically complex regions that are more informative in detecting potential abnormalities.

The activation around the wrist indicates that the model is not relying on background patterns or irrelevant regions of the image when coming to a conclusion. It seems to evaluate meaningful features from areas where abnormalities commonly occur. The lack of focus around the length of the forearm bone support's the model's judgement that no abnormalities are present in this image.

This example demonstrates that EfficientNet-B0 is capable of identifying regions that are relevant even when a dataset label may refer to a wider body part category. By focusing on the wrist, the model appears to evaluate areas with the highest potential clinical significance before determining that no abnormality exists.

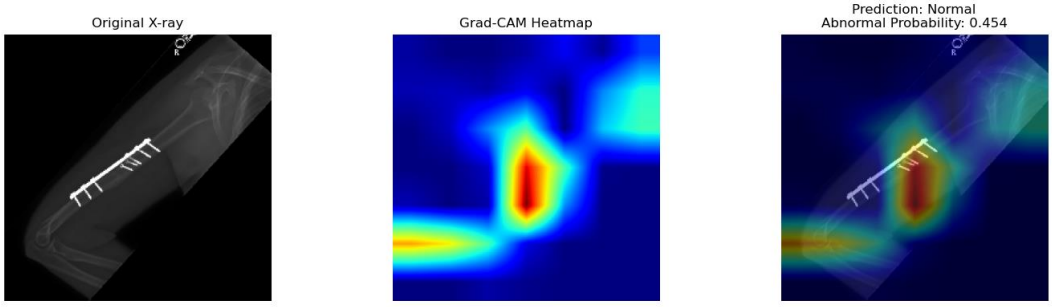


Figure 6.10: Misclassification (EfficientNet-B0)

Figure 6.10 shows a failure case for EfficientNet-B0. This image originates from a 'XR_HUMERUS' study that with a true label of abnormal, however the model predicts this image

is a normal case. The abnormal probability assigned in 0.454 representing a low confidence in accuracy. By looking at the heatmap, we can infer that the model detected some features that partially resemble abnormal patterns but did not collect enough evidence to classify the whole image as abnormal.

The heatmap reveals that the model's attention is mostly concentrated around the central region of the humerus rather than on the likely metallic support piece visible in the image. The screws and supporting structure form important elements that might be expected to influence the abnormal classification. However, the activation map indicates that the model mostly ignores this and appears to focus its concentration on surrounding regions.
This behaviour highlights a limitation in the representations learned by the network. Despite the presence of clearly visible supporting hardware the model's attention does not seem to align with these elements. As a result, the model fails to capture the contextual cues that could indicate the abnormal nature of this image.

The moderate probability of 0.454 that the image is abnormal can suggest that the model is uncertain rather than confident in its incorrect classification. This borderline prediction indicates that some abnormal signal does exist but the final choice to classify this image as a normal case falls just below. This is an example of how subtle shifts in feature weighting can influence final outputs.

In conclusion, this failure demonstrates that EfficientNet-B0 may not always attend to the most visually obvious structures present in the image.

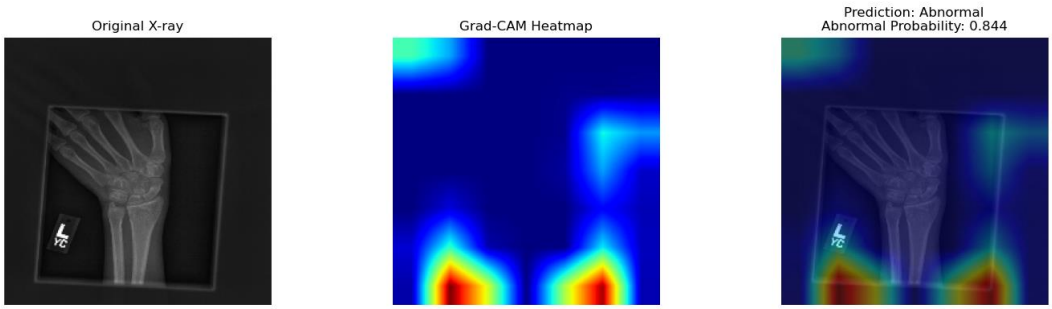


Figure 6.11: Correct Abnormal Classification (ResNet50)

Figure 6.11 shows a correctly classified abnormal wrist image. The assigned abnormality probability is 0.844, indicative of a high confidence prediction. Albeit a correct classification, this case is interesting because of the discrepancy between the prediction and the focus of the model.

The activation map shows that the strongest activate is concentrated around the lower edges of the image rather than on the wrist bone structure. In particular this heatmap highlights the peripheral region close to the image's borders. This tells us that the model's prediction is influenced by features outside the main intended diagnostic area.

This suggests that that the model might rely on image patterns rather than assessing the structures associated with abnormality. Elements such as borders, positioning of artefacts or background intensities can sometimes correlate with abnormality, creating unintended predictive cues during the training process.

This highlights a potential weakness in the model's reasoning process. The lack of attention on the wrist joint itself speaks to the fact that the prediction was not derived from the region it should be derived from. As a result, the correctness of this prediction might be partly coincidental or a result of overfitting.

This example illustrates an important limitation in CNN predictions within the medical field. The model appears to achieve a correct classification while relying on features that are not related to the existing abnormality, revealing that correctness does not necessarily imply that the model is carrying out its task as intended.

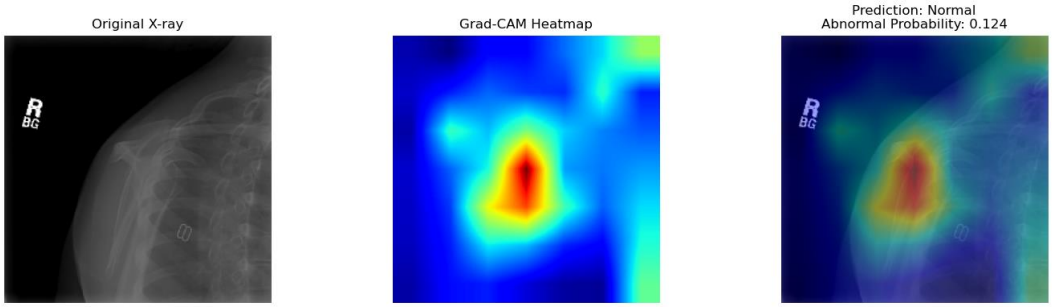


Figure 6.12: Correct Normal Classification (ResNet50)

Figure 6.12 shows a correctly classified normal study. The abnormality probability is 0.124, displaying a high confidence that the image belongs to the normal class.

The heatmap shows a fair and spread out activation across the central regions of the shoulder area with no unexpected focus on any features other than the intended shoulder region. Unlike abnormal cases where activation is usually focused around visually distinctive elements, the attention appears more distributed around the focus region for this image. This displays

consistent normal classification behaviour since the model does not detect highly discriminative patterns that would point to an abnormal case.

From an interpretability standpoint, this example is mostly mundane, this is informative in itself. The absence of strong activation suggests that the model does not rely on isolate visual artefacts when identifying normal cases. Instead, the model appears to evaluate a wider structural consistency across the image before determining that abnormal features are not present.

It is important to notice the small regions of focus outside of the exact shoulder region present near the neck and ribcage area. From this it is plausible to deduct that the model might have had a higher confidence score if those exterior regions were not explored by the model.
The relatively low abnormality probability with the lack of sharply focused activation other than the target region indicate a stable normal class prediction.

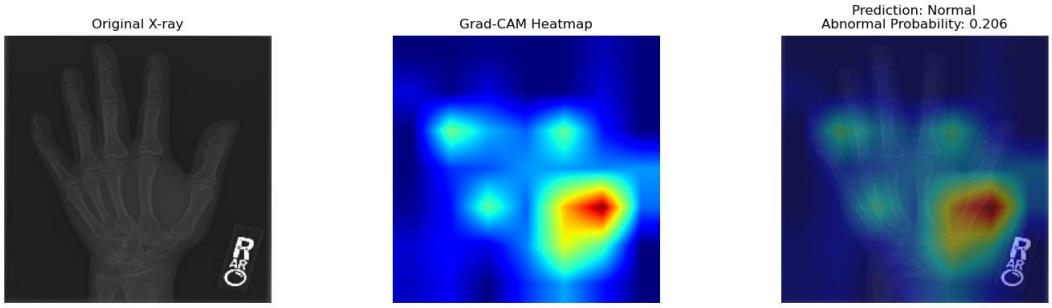


Figure 6.13: Misclassification (ResNet50)

Figure 6.13 shows a misclassification produced by ResNet50, the radiograph originates from 'XR_HAND' with a true study label of positive or abnormal, however the model predicts this case to be of the normal class. The model has a low abnormal probability of 0.206 meaning it is approximately 80% confident in its incorrect classification.

The heatmap shows the model's attention is concentrated around the right side of the hand. Activation is visible across a number of finger joints, but these regions receive significantly weaker emphasis. The distribution of the heatmap indicates that the model is not predominantly focused on the central bone structure where discriminative features for abnormality may be present.

This may suggest that the model's feature extraction is dictated by localised image characteristics rather than the wider region of the hand. The strong concentration of the heatmap near the watermarked image marker and surrounding areas can imply that contextual or positional cues may contribute to the decision process. We can infer that the model appears to prioritise features that are not relevant when differentiating between abnormal and normal cases.

In contrast to the EfficientNet-B0 failure case which produced borderline predictions close to the decision threshold, this decision shows a more decisive misclassification. The low abnormal probability tells us that the extracted features align with patterns the model associates with normal cases, in turning telling us that features in the image were not properly captured within the learned feature representation.

Overall, this misclassification illustrates a limitation in ResNet50's attention behaviour. It has shown that focus does not clearly align with the most informative regions of the image, leading to confident misclassification.

6.8.1 Grad-CAM Conclusions

Overall, Grad-CAMs provide a qualitative analysis into how each architecture uses spatial features when performing abnormality classification. In a majority of cases, EfficientNet-B0 demonstrates more localised attention patterns around the intended anatomical regions, this lies in consistency with its stronger abnormal detection performance observed in the quantitative evaluations. However, the examples above reveal that both models rely on contextual markers occasionally, this includes but is not limited to medical supports, peripheral image structures, positional markers, etc. This behaviour becomes particularly evident in certain misclassification cases where a model's attention is directed away from the more informative regions of an image. When paired with the experimental results from earlier, these visualisations tell us that although both architectures learn useful, context relevant information, their decision choices do not always follow suite, their decision processes are not always aligned with clinically relevant expectations/ structures. The Grad-CAM study provides additional insight by revealing potential limitations in model reasoning that are not always captured by performance metrics alone, making this an important part of the comparative analysis in this dissertation.

6.9 Final Test Set Evaluation

The final stage of evaluation in this study was conducted using the held out test subset to assess whether performance trends seen during training and validation translate to completely

untouched and unseen data. The test set was not used at any stage during development, allowing for an unbiased estimate of final model performance. The objective of this subsection is to determine whether the behaviours observed earlier; EfficientNet-B0's slightly stronger abnormal detection performance paired with its stability across all seeds and dataset splits. This trend remains consistent even when evaluated on new data. Figure 6.14 presents the study level performance across evaluation metrics like abnormal recall, abnormal precision, F1 score and overall accuracy.

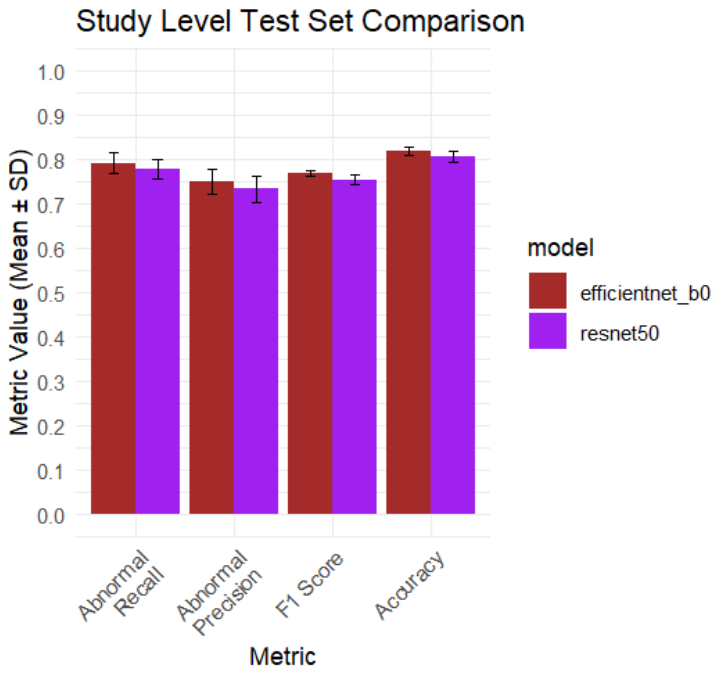


Figure 6.14: Key Metrics at Study Level for Both Models (test subset)

The results show that EfficientNet-B0 achieves higher performance on the above metrics when evaluated on unseen data, however the performance is only slightly better. The most relevant metric for this task; abnormal recall tells us that EfficientNet-B0 detects abnormal studies slightly more effectively than ResNet50. This observation is consistent with the earlier validation experiments, suggesting that the model maintains its advantage when evaluated on the held out test set.

Abnormal precision follows the same pattern with EfficientNet-B0 producing slightly higher scores, validating the model's previously discussed advantage in abnormality detection but it also produces fewer false positive predictions in comparison with ResNet50. Once again, a similar trend is reflected in the F1 score, which combines precision and recall, which provides a more balanced measure of classification performance. The marginally higher F1 score further supports the conclusion that the architecture offers a small but fair improvement in overall abnormality detection.

It is important to note that the error bars that represent the standard deviation across all runs remain relatively small for all metrics.

Taken together the test results confirm that the performance patterns observed during training and validation persist on unseen data as well. Both architectures achieve mostly similar performance levels. The standard deviation values also tell us that model behaviour is not strongly influenced by the choice of seeds or dataset split, this supports the reliability of comparative conclusions draws throughout analysis.

One observation of this subsection is that both architectures show a small improvement in study level abnormal recall on the test set in comparison to testing and validation performance. ResNet50 improves by around 0.0214 and EfficientNet-B0 improves by around 0.0256. Usually, completely unseen test data would be expected to produce slightly worse or in the best case similar results to validation. There are several explanations for this, firstly, despite patient level splitting to control data leakage, the held out test set using the fixed dataset shuffle seed of 67 might have created a partition that is more favourable for class balance or image quality distribution than the validation set used during training. Another reason might be that the checkpoint selection mechanism might have selected model states that generalise marginally better than they are able to validate. Thirdly, with only six test runs per model, the improvement may fall within the natural variability of the experimental setup rather than being because of genuine model design. Putting these elements together, these explanation suggest that the improvements in the test set should be interpreted carefully rather than solid evidence of a better ability to generalise. It is more likely a reflection of dataset split composition and the small scale of this experimental framework rather than a meaningful architectural find. This is an important observation to take into account rather than moving to the straightforward conclusion that both models generalise well on unseen data.

The absence of a meaningful performance drop when moving from validation to unseen test set suggests that the training configuration did not lead any significant oversitting. In particular, the stable abnormal recall values demonstrate that both models retain a similar ability to detect abnormal studies when applied to unseen data. Overall, EfficientNet-B0 maintains a modest advantage over ResNet50.

6.10 Summary of Key Findings

The experimental results seen across Section 6 give us a number of key findings with regard to the comparative behaviour of 2 CNNs evaluated in this study. Across validation and testing,

EfficientNet-B0 shows a small but stable performance advantage over ResNet50. This improvement is reflected across all relevant study metrics, from this we can conclude that the design of EfficientNet-B0 enables a more effective feature extraction process for musculoskeletal radiograph image classification, given the experimental conditions used in this dissertation.

Emphasis was placed on the detection of abnormal studies, lying in accordance with clinical practice. Although EfficientNet-B0 achieved higher scores it was only by a small margin, this advantage was observed throughout all experiments carried out, cementing the fact that observed improvements reflects a genuine architectural characteristic rather than random variation.

The relatively small standard deviation values indicate that the achieved results are reproducible and not adversely affected by stochastic factors in training or dataset shuffling. This stability strengthens the reliability of the comparison and supports the validity of the methodologies employed in this study.
The Grad-CAM analysis provides a more qualitative interpretation and allows us to zoom into individual examples for each model rather than just interpreting metric scores. We can see that both models utilise spatial features when coming to their final predictions, supporting the correctness of predictions and indicating that their task is carried out as intended. Some Grad CAM visualisations tell us that both architectures occasionally rely on task irrelevant cues rather than purely anatomical features as the task intends. Despite strong quantitative performances, Grad CAMs allow us to see how decision processes are not always aligned as intended.

It is important to put these findings in context with existing work, the accuracy values achieved by both models in this study are generally consistent with the single most singular model approaches. (Teeyapan, 2021) reported a Cohen's Kappa of 0.717 for EfficientNet-B3, (Ananda, et al., 2021) reported a mean accuracy of 0.857 for ResNet50 on wrist radiographs. The study level accuracy values of 0.82 for EfficientNet-B0 and 0.81for ResNet50 are comparable, though direct comparison and definitive conclusions are limited by differences in evaluation metrics, dataset splits and study scope. (He, et al., 2021) shows that ResNet performs competitively given the significant variations depending on body part, this pattern is consistent with the seed instability observed for ResNet50 in this study, which may indicate that architectural sensitivity to initialisation may partly reflect the variety of MURA rather than stochastic training effects.

In conclusion, both architectures perform their task to identify abnormal cases effectively with EfficientNet-B0 possessing a minimal advantage in overall performance. This lies in accordance with literature and the Abstract outlining EfficientNet-B0's strengths since belonging to a more revised and modern CNN family.

7 Conclusions

7.1 Main Conclusion

The purpose of this dissertation was to conduct a controlled comparative analysis of 2 CNN architectures, ResNet50 and EfficientNet-B0 used on the MURA dataset and investigating how differences in architectural design influence abnormality detection performance, model stability and interpretability under identical training conditions. The objective was never to build a real useable diagnostic tool or to identify the best architecture. The aim was analytical and to investigate how differences in architectural design manifests as differences in behaviour when both models are trained under the same conditions and using the same pipelines. The most important conclusion drawn from this study is that there is no definitive "winner" in terms of better architecture, relative to this experimental setting. EfficientNet-B0 demonstrates a consistent but minimal advantage in study level abnormal recall. The minimal advantage is approximately 1.4% points on the held out test set, this model also produces more stable outcomes across seeds and dataset splits. However, the margin is far too narrow and the scale of experiments in this dissertation are too limited to support a strong or conclusive claim of architectural dominance.

It is also important to note that both architectures fall short of the performance levels that would be required to even consider real world clinical deployment. Abnormal recall values in the range of 0.75 to 0.81 at the study level imply that between 20 and 25 percent of abnormal cases would be entirely missed. In real world terms, this would mean that 1 in 5 people could potentially miss out on important medical care. Recent literature has attempted to achieve higher accuracy values through task specific and purpose built framework rather than implementation via general architectural advances. An example of this is FracNet achieving near perfect scores, albeit specifically on fracture detection rather than across the wide variety of MSK abnormalities (Alwzwazy, et al., 2025), this further supports the view that generalised but robust abnormality detection remains an open challenge.

The contribution of this dissertation is not a recommendation of one architecture over another, but a structured insight into how 2 different design principles behave differently under the

same controlled conditions. This finding shows genuine value for guiding further and more advanced comparative work in this field of machine learning.

7.2 Objectives Review

Reflecting on the 6 objectives from Section 1. Objectives 1, 2 and 3 were met most completely. In all, a thorough background investigation was conducted, an experimental framework that was controlled was implemented and both models were trained and evaluated under identical conditions using transfer learning and ImageNet pretrained weights.

The strict design of patient level partitioning, fixed seeds and 2 dataset splits along with checkpoint selection provides a reliable foundation from which conclusions can be drawn for the given conditions which allows for a reasonable level of confidence.

Objective 4 was only met partially, this is because only 2 split ratios were tested throughout this study, meaning that conclusions drawn about how the amount of data in each subset influences generalisation in architectures remain more informative rather than conclusive. Objective 5 was met, however the emphasis on abnormal recall as the main metric means that other metrics were reported but not to the same extent and depth.

Objective 6 was partially met as well. The Grad-CAM analysis consisted of 6 heatmaps, 3 for each architecture. While these heatmaps did provide genuine insight into understanding what areas of a given image influenced final decisions the most, this analysis cannot be considered as representative samples of model behaviour across a dataset of more than 40,000 images. There is no established basis on how many visualisations constitute a sufficient number of heatmaps for absolute interpretability conclusions in large medical imaging datasets. However, a better strategy might be sampling image by body part, this might have created a stronger prediction confidence and classification outcomes based on this strategy may have produced more conclusive and defensible analysis.

7.3 Limitations

Three reporting strategies were considered during analysis; one strategy was to select the single best run, another was to report mean performance only and the final consideration was to only analyse worst case outcomes. Each of these considerations carried a meaningful flaw. Reporting the best single run would constitute cherry picking and mean that the natural variance present in deep learning would be entirely misinterpreted. Reporting the mean alone seemed like a more honest approach however it would blur the instability that was observed

especially in ResNet50 across seeds. This is important to note because instability is a relevant finding in any realm of machine learning and holds a heavier emphasis in a medical context. If this dissertation only reported worst case performances, it would be a conservative and safety oriented way to analyse results. However, it risked being dramatic and detached from the main purpose of this study; to understand model behaviour under said conditions.
The approach taken in this dissertation included reporting clinically relevant metrics as means with standard deviations across multiple seeds and splits. This approach attempts to find balance between the 3 reporting strategies that were originally considered, but this approach is also limited by the minimal number of runs.

Another important limitation stems from the improvement observed from the validation testing to the test set for both models. This suggests that the design of the held out test set under the fixed dataset shuffle seed might have introduced some favour going into the final evaluation. Paired with only 12 controlled runs, the conclusions drawn should be understood as structured observations under certain conditions rather than generalisable findings.

7.4 Future Work

The first expansion to this project would be conducting more experimental runs through more seeds and split configurations, allowing higher confidence in conclusions drawn and strengthening the reliability of model comparisons. It would also be beneficial to have a systematic Grad-CAM sampling strategy to analyse findings by body part to draw more specific conclusions, it would take qualitative analysis from illustrative to analytically meaningful.
In terms of aggregation, comparing the max rule to mean probability thresholding or majority voting would be a means of clarifying how much of the study level performance seen in this study is a result of architectural design rather than strict aggregation design.

As a long term goal, Vision Transformers should be added to this comparison, this represents a different approach to feature extraction through self-attention rather than convolutions. This would create a wider landscape that includes ResNet and EfficientNet models in assessing the most appropriate methods to detect abnormal studies.

The main question of this dissertation that cannot be fully answered is whether minimal but consistent advantages of parameter conscious architecture like EfficientNet-B0 hold across different medical imaging settings and larger scale experiments. This remains an open question and represents the most valuable direction for further research.

8 References

GBD 2021 Other Musculoskeletal Disorders Collaborators, 2023. Global, regional, and national burden of other musculoskeletal disorders, 1990–2020, and projections to 2050: a systematic analysis of the Global Burden of Disease Study 2021. *The Lancet Rheumatology,* 5(11), pp. 670-682.

Ahmed, T. & Sabab, N. H. N., 2022. Classification and understanding of cloud structures via satellite images with EfficientUNet. *SN Computer Sciene ,* 3(1), pp. 1-11.

Alwzwazy, H. A., Alzubaidi, L., Zhao, Z. & Gu, Y., 2025. FracNet: An end-to-end deep learning framework for bone fracture detection. *Pattern Recognition Letters,* Volume 190, pp. 1-7.

Ananda, A. et al., 2021. Classification and visualisation of normal and abnormal radiographs; a comparison between eleven convolutional neural network architectures. *Sensors,* 21(16), p. 5381.

Azam, S. et al., 2023. Using Feature Maps to Unpack the CNN 'Black Box' Theory with Two Medical Datasets of Different Modality. *Intelligent Systems with Applications ,* Volume 18, p. 200233.

Babaei, H., Zamani, M. & Mohammadi, S., 2025. The impact of data splitting methods on machine learning models: A case study for predicting concrete workability. *Machine Learning for Computational Science and Engineering,* 1(1), p. 21.

Bengio, Y., Simard, P. & Frasconi, P., 1994. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks,* 5(2), pp. 157-166.

Bontempo, A. C., Bontempo, J. M. & Duberstein, P. R., 2025. Ignored, Dismissed, and Minimized: Understanding the Harmful Consequences of Invalidation in Health Care - A Systematic Meta-Synthesis of Qualitative Research. *Psychological Bulletin,* 151(4), pp. 399-427.

Bouthillier, X. et al., 2021. *Accounting For Variance in Machine Learning Benchmarks.* San Jose, ML Systems Conference.

Buckland, M. & Gey, F., 1994. The Relationship Between Recall and Precision. *Journal of the American Society for Information Science,* 45(1), pp. 12-19.

Burt, T. et al., 2017. The Burden of the "False-Negatives" in Clinical Development: Analyses of Current and Alternative Scenarios and Corrective Measures. *Clinical and Translational Science,* 10(6), pp. 470-479.

Deng, J. et al., 2009. *ImageNet: A large-scale hierarchical image database.* Miami, 2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Diaz, E., Lakshminarayanan, V. & Benavides, L. C., 2026. Integrating Explainability and Bias Detection in Binary Medical Image Classification: A Systematic Review. *Machine Learning: Health,* 2(1), p. 011001.

Geirhos, R. et al., 2020. Shortcut Learning in Deep Neural Networks. *Natural Machine Intelligence ,* 2(11), pp. 665-673.

Goodfellow, I., Bengio, Y. & Courville, A., 2016 . *Deep Learning.* Cambridge for MIT Press: MIT Press.

Gulbahor, A., 2025. Comparative Efficiency Analysis of CNN, ResNet and EfficientNet in Medical Image Classification. *Miasto Przyszłości,* 66(N/A), pp. 185-186.

He, K., Zhang, X., Ren, S. & Sun, J., 2015. *Deep Residual Learning for Image Recognition.* Las Vegas, NV, USA, IEEE conference on computer vision and pattern recognition.

He, M., Wang, X. & Zhao, Y., 2021. A calibrated deep learning ensemble for abnormality detection in musculoskeletal radiographs. *Scientific Reports,* 11(1), p. 9097.

Hicks, S. A. et al., 2022. On Evaluation Metrics for Medical Applications of Aritificial Intelligence. *Scientific Reports,* 12(1), p. 5979.

Kailani, N. M. S., 2025. *Comparison Between ResNet and EfficientNet for Image Classification -An Analytical Study of Performance and Efficiency,* Libya: The North African Journal of Scientific Publishing (NAJSP).

Kingma, D. P. & Ba, J., 2015. *Adam: A Method for Stochastic Optimization.* San Diego, CA, 3rd International Conference on Learning Representations (ICLR 2015).

Kumar, V. et al., 2024. Unified Deep Learning Model for Enhanced Lung Cancer Prediction with ResNet-50-101 and EfficientNet-B3 using DICOM Images. *BMC Medical Imaging ,* 24(1), p. 63.

LeCun, Y., Bengio, Y. & Hinton, G., 2015. Deep Learning. *Nature,* 521(7553), pp. 436-444.

Lin, A. et al., 2024. *Convolutional Neural Networks in Medical Imaging: A Review,* Singapore: Springer.

Litjens, G. et al., 2017. A Survey on Deep Learning in Medical Imaging Analysis. *Medical Image Analysis ,* Volume 42, pp. 60-88.

McCarthy, E. & Frassica, F., 2014. The Pathophysiology of Fractures. In: E. F. M. a. F. J. Frassica, ed. *Pathology of Bone and Joint Disorders: With Clinical and Radiographic Correlation.* Cambridge: Cambridge University Press, pp. 30-46.

Mienye, I. D. et al., 2025. Deep Convolutional Neural Networks in Medical Image Analysis: A Review. *InformatioM by MDPI,* 16(3), p. 195.

Muhammad, D. & Bendechache, M., 2024. Unveiling the Black Box: A Systematic Review of Explainable Artificial Intelligence in Medical Image Analysis. *Computational and Structural Biotechnology Journal (,* 24(2), pp. 235-252.

Pascanu, R., Mikolov, T. & Bengio, Y., 2013. *On the Difficulty of Training Recurrent Neural Networks.* Atlanta, Georgia, PMLR (Proceedings of Machine Learning Research).

Paszke, A. et al., 2019. *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* Vancouver, Advances in Neural Information Processing Systems (NeurIPS).

Prechelt, L., 1998. Early Stopping - But When?. In: G. B. O. a. K. Müller., ed. *Neural Networks: Tricks of the Trade.* Berlin and Heidelberg: Springer-Verlag, pp. 55-69.

Raghu, M., Zhang, C., Kleinberg, J. M. & Bengio, S., 2019. *Transfusion: Understanding Transfer Learning for Medical Imaging.* Vancouver, Curran Associates Inc.

Rajpurkar, P. et al., 2018. *MURA: Large Dataset for Abnormality Detection in Musculoskeletal Radiographs.* Amsterdam, 1st Conference on Medical Imaging with Deep Learning.

Rosenkranz, M., Kalina, K. A., Brummund, J. & Kastner, M., 2023. A Comparative Study on Different Neural Network Architectures to Model Inelasticity. *International Journal for Numerical Methods in Engineering,* 124(21), pp. 4802-4840.

Sandag, G. A. & Kabo, D. T., 2024. Comparative Analysis of Lung Cancer Classification Models Using EfficientNet and ResNet on CT-Scan Lung Images. *CogITo Smart Journal,* 10(1), pp. 259-270.

Schneider, L., Bischl, B. & Feurer, M., 2025. *Overtuning in Hyperparameter Optimization.* New York, Proceedings of Machine Learning Research (PMLR).

Selvaraju, R. R. et al., 2017. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.* Venice, IEEE International Conference on Computer Vision (ICCV).

Shorten, C. & Khoshgoftaar, T. M., 2019. A survey on Image Data Augmentation for Deep Learning. *Journal of Big Data,* 6(1), pp. 1-48.

Singh, V. et al., 2021. Impact of Train/Test Sample Regimen on Performance Estimate Stability of Machine Learning in Cardiovascular Imaging. *Scientific Reports ,* 11(1), p. 14490.

Tan, M. & Le, Q. V., 2019. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.* Long Beach, CA, Proceedings of the 36th International Conference on Machine Learning (ICML).

Teeyapan, K., 2021. *Abnormality Detection in Musculoskeletal Radiographs using EfficientNets.* Phitsanulok, Thailand, 2020 24th International Computer Science and Engineering Conference (ICSEC).

Teng, Q. et al., 2022. A Survey on the Intpretability of Deep Learning in Medical Diagnosis. *Multimedia Systems,* 28(6), pp. 2335-2355.

The Royal College of Radiologists , 2023. *Clinical radiology UK workforce census 2022 report,* London: Royal College of Radiologists.

The Royal College of Radiologists, 2018. *Standards for Interpretation and Reporting of Imaging Investigations,* London: The Royal College of Radiologists.

Veličković, P. et al., 2018 . *Graph Attention Networks.* 6th International Conference on Learning Representations (ICLR), Openreview.net.

Vina, A., 2025. *What is ResNet-50 and What Is Its Relevance in Computer Vision.* [Online] Available at: https://www.ultralytics.com/blog/what-is-resnet-50-and-what-is-its-relevance-in-computer-vision
[Accessed 24 01 2026].

Wegmeth, L., Vente, T., Purucker, L. & Beel, J., 2023. *The Effect of Random Seeds for Data Splitting on Recommendation Accuracy.* Singapore, Proceedings of the 3rd Workshop on Perspectives on the Evaluation of Recommender Systems (Perspectives@RecSys 2023).

Yang, J. et al., 2022. *Do deep neural networks always perform better when eating more data?,* Ithaca, NY (arXiv host): arXiv PrePrint.

Appendix A Personal Reflection (Compulsory)

A.1 Reflection on Project

For the most part this study was executed as planned, the experimental framework performed as intended once early pipeline issues were resolved. The most important processing issues encountered was handling data. Experimental results were extracted manually from the terminal output from VSCode and pasted into ChatGPT, individual metrics were isolated and then copied into Excel for analysis. Figure 8.1 shows values are received from the IDE terminal, Figure 8.2 shows exactly how ChatGPT assisted in the manual extraction process in this study.

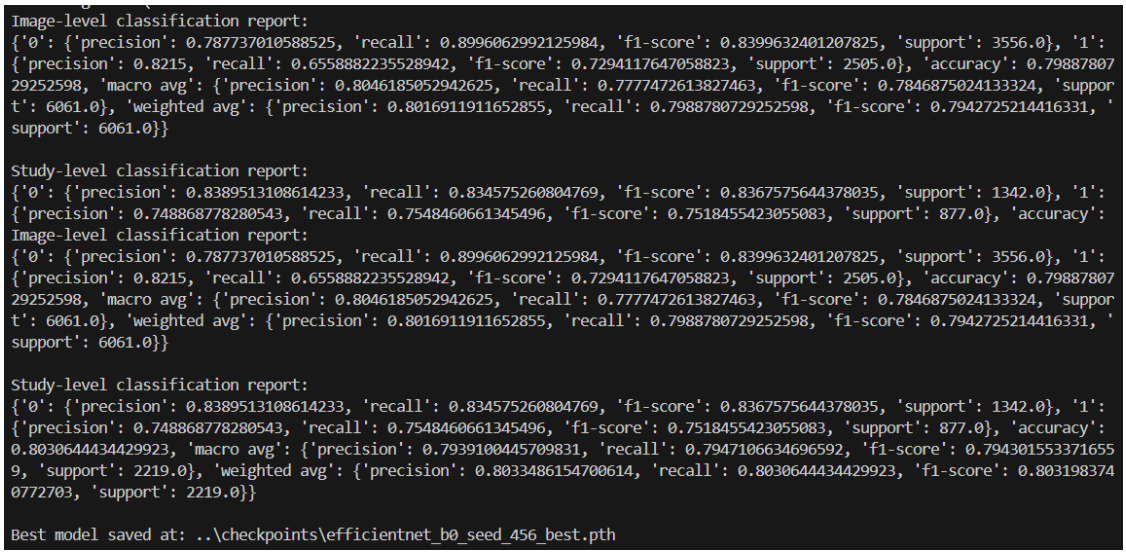


Figure 8.1: Raw Outputs from Training Script

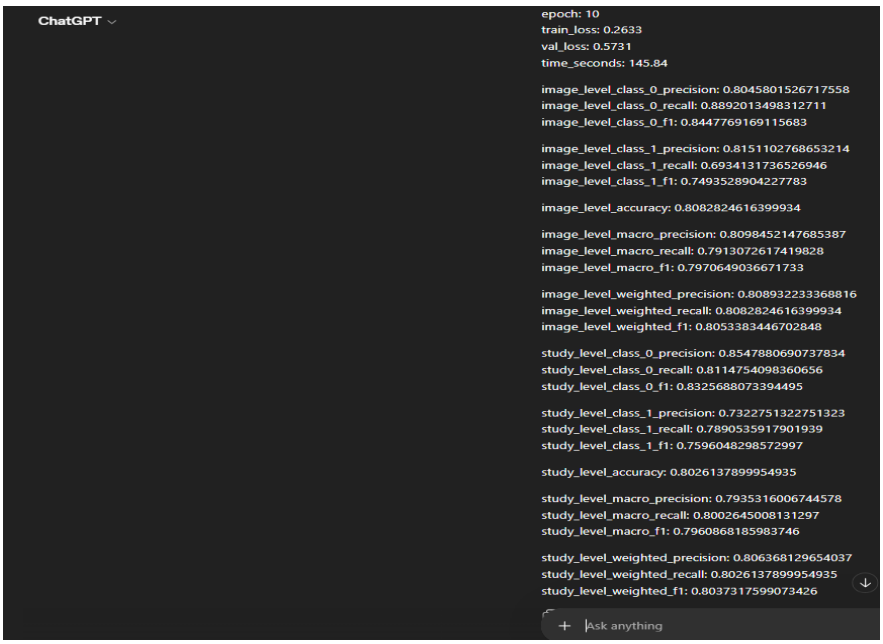


Figure 8.2: ChatGPT Results Collection

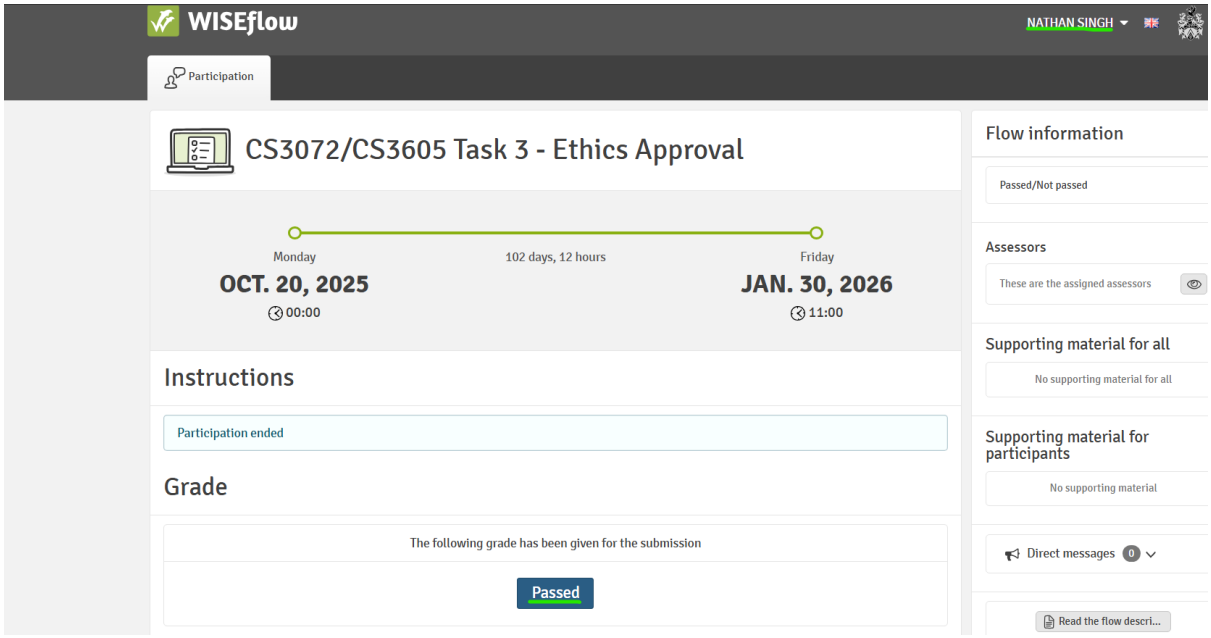
With 151 rows and over 35 columns per entry, this processed consumed a significant amount of time that could have been better spent on deeper analysis. An automated logging system that wrote metrics directly to the Excel files during training would have eliminated this problem entirely. The decision to create one was pursued but eventually scrapped in order to keep the project focused on architectural analysis rather than developing a tool. Looking at it now, this lightweight and efficient solution might have been worth the time.
Another issue was inconsistent naming conventions across Excel sheets, labels were not labelled as uniformly as they should be. This caused repeated debugging issues during statistical analyses in RStudio, small spelling inconsistencies and symbol mismatches produced errors that were time consuming to resolve. Establishing a simple and consistent naming strategy before data collection begins is an important lesson learned that will be applied to any future work.
It is important to reiterate that AI assistance was used only for data extraction, ChatGPT was used to convert terminal output into ordered numerical values for manual entry into Excel sheets. AI was not used in analysis, interpretations or any written content.

A.2 Personal Reflection

If this project was done again, there would be a greater emphasis on increasing the number of runs. The conclusions drawn in this dissertation are directional and consistent but a larger set of runs would provide more analytical depth that would have meaningfully built on the strength of this study. The trade-off between scale of experimentation and depth of analysis was a large factor in this dissertation, resolving these issues earlier would have allowed for more time to do both properly.

Appendix B Ethics Documentation (Compulsory)

B.1 Ethics Approval



Appendix C Video Demonstration (Compulsory)

C.1 Link to the video

Appendix D Other Appendices

More relevant material

N/A.